

UNIVERSIDAD POLITÉCNICA SALESIANA

Unidad de Posgrados

Maestría en Software

Asignatura: Patrones de Diseño de APIs

INFORME FINAL DEL PROYECTO

Billetera de Ahorro

Plataforma de planes de ahorro programado dirigida por APIs

Docente: Ing. Patsy Malena Prieto, MSc.

Integrantes: _____

Ecuador, octubre de 2026

Resumen

Este informe documenta el diseño y la construcción de la *Billetera de Ahorro*, un producto bancario de planes de ahorro programado expuesto mediante APIs. El cliente fija una meta y una fecha objetivo, el sistema calcula la cuota mensual, la debita de forma automática de su cuenta y muestra el progreso; si el cliente bloquea el plan, obtiene una tasa preferencial.

El trabajo siguió las cuatro fases de la asignatura. En la Fase 1 justificamos el caso de negocio con un modelo de monetización indirecta basado en el spread y la permanencia de los depósitos. En la Fase 2 elegimos un estilo híbrido REST y orientado a eventos, desplegado como monolito modular detrás de un API Gateway, y justificamos sus patrones con la matriz ISO/IEC 25010. En la Fase 3 escribimos el contrato OpenAPI 3.1 antes que el código (contract-first). En la Fase 4 implementamos el backend en Node.js por capas, con JWT RS256, Circuit Breaker, Retry con jitter, Transactional Outbox e idempotencia; alcanzamos 97,9 % de cobertura de líneas con 97 pruebas, desplegamos en AWS con un pipeline de CI/CD y medimos el sistema con k6: bajo carga sostenida de 200 usuarios el p95 fue de 336 ms con 0,001 % de errores, y el punto de ruptura se ubicó en unas 220 peticiones por segundo, limitado por la base de datos.

Palabras clave: API-First, OpenAPI, arquitectura orientada a eventos, patrones de resiliencia, JWT, pruebas de carga, CI/CD.

Índice general

Resumen	1
1. Introducción	1
1.1. Contexto y problema	1
1.2. Objetivos	1
1.3. Alcance	1
1.4. Organización del repositorio y del documento	2
2. Fase 1: justificación de negocio	3
2.1. El problema	3
2.2. Qué hace el producto	3
2.3. Objetivos de las APIs	4
2.4. La cadena de valor de las APIs	5
2.5. Monetización	6
2.6. Ecosistema de APIs	6
2.7. Anatomía de la API	7
2.8. Naturaleza de la API	8
2.9. Indicadores	9
2.10. Riesgos	9
2.11. Qué queda por decidir	10
2.12. Requerimientos del producto	10
3. Fase 2: arquitectura y patrones	13
3.1. Introducción	13
3.2. Arquitectura de software	14
3.3. Selección del estilo arquitectónico	17
3.4. Patrones de diseño	19
3.5. Conclusiones	24

4. Fase 3: modelo de datos y contrato de la API	25
4.1. Enfoque contract-first	25
4.2. Convenciones del contrato	26
4.3. Modelo de datos orientado a la API	26
4.4. Operaciones del contrato	27
4.5. Ajuste del contrato al frontend	29
4.6. Validación	29
5. Fase 4: desarrollo, seguridad y despliegue	30
5.1. Introducción	30
5.2. Implementación del backend	30
5.3. Seguridad	32
5.4. Pruebas unitarias y de integración	32
5.5. Despliegue y CI/CD	33
5.6. Pruebas de carga y estrés (k6)	34
5.7. Evidencias	41
6. Integración con el frontend y pruebas de extremo a extremo	42
7. Conclusiones y trabajo futuro	43
7.1. Conclusiones	43
7.2. Trabajo futuro	43
Referencias	44
A. Detalle de los patrones de diseño	45
A.1. Adapter	45
A.2. Decorator	45
A.3. Strategy	46
A.4. State	46
A.5. Observer (eventos de dominio)	47
A.6. Requisito para aplicar State y Strategy en el Batch	48
B. Evidencias	49
B.1. Validación del contrato	49
B.2. Pruebas unitarias y de integración	49
B.3. Seguridad contra la API desplegada	53

B.4. Despliegue en AWS	57
B.5. Pipeline de CI/CD	59
C. Distribución del trabajo	61

Índice de figuras

3.1. Diagrama de contexto (C4, nivel 1)	15
3.2. Diagrama de contenedores (C4, nivel 2)	16
3.3. Vista ArchiMate de la Fase 2	18
4.1. Contrato publicado con Swagger UI en el entorno de AWS	25
5.1. Throughput y usuarios virtuales	35
5.2. Tiempo de respuesta	36
5.3. Recursos del servidor	36
5.4. Tiempo de respuesta en el pico	37
5.5. Recursos en el pico	38
5.6. Punto de ruptura	39
5.7. Latencia hacia la ruptura	40
5.8. Recursos hacia la ruptura	40

Índice de tablas

1.1. Organización del repositorio	2
3.1. Requisitos con impacto en la arquitectura	13
3.2. Capas y componentes de la solución	14
3.3. Contenedores del sistema	16
3.4. Comparación de estilos según ISO/IEC 25010	17
3.5. Patrones aplicados	19
3.6. Resiliencia según el camino hacia el Core	21
3.7. Patrones de diseño a nivel de código	22
3.8. Eventos del sistema	23
3.9. Patrones descartados	24
4.1. Convenciones de diseño del contrato	26
4.2. Recursos del modelo de la API	27
4.3. Operaciones del contrato OpenAPI (generada desde contracts/openapi.yaml)	27
5.1. Capas del backend	30
5.2. Patrones implementados	31
5.3. Mecanismos de seguridad	32
5.4. Cobertura de código (umbral configurado: 80 %)	33
5.5. Recursos en AWS	33
5.6. KPIs de la carga sostenida	34
5.7. Tiempos por endpoint en la carga sostenida	35
5.8. KPIs de la prueba de pico	37
5.9. Evolución hacia el punto de ruptura	38
5.10. Evidencias	41
C.1. Distribución del trabajo por integrante	61

CAPÍTULO 1

Introducción

1.1 Contexto y problema

El ahorro de las personas suele ser un residuo: se ahorra lo que sobra a fin de mes. Los productos tradicionales piden montos mínimos, plazos rígidos y trámites en agencia, y el cliente no sabe cuánto tendrá al final. Del lado del banco, el depósito a la vista es barato pero volátil, y la apertura de productos de ahorro en canales digitales se abandona sin que el banco sepa por qué.

La *Billetera de Ahorro* responde a ambos problemas: automatiza el aporte, muestra desde el primer momento cuánto dinero tendrá el cliente y no exige monto mínimo. A cambio, el banco obtiene depósitos recurrentes y de mayor permanencia. El producto se construyó con un enfoque API-First: la capacidad de ahorro se expone como un contrato que hoy consume la banca web y mañana podrían consumir socios externos.

1.2 Objetivos

Objetivo general. Diseñar, implementar y desplegar una plataforma de planes de ahorro programado dirigida por APIs, aplicando los patrones de diseño y arquitectura vistos en la asignatura.

Objetivos específicos.

- Justificar el caso de negocio, la cadena de valor y el modelo de monetización de la API (Fase 1).
- Seleccionar y justificar el estilo arquitectónico y los patrones de diseño (Fase 2).
- Especificar el modelo de datos y el contrato de la API con OpenAPI (Fase 3).
- Implementar los servicios con seguridad, pruebas, pruebas de carga y despliegue automatizado (Fase 4).

1.3 Alcance

El alcance aprobado por el equipo incluye la creación de varios planes con cuota fija mensual, la simulación pública, el débito automático con cinco intentos, los aportes y retiros, el bloqueo y desbloqueo del plan, la cancelación con devolución de fondos y la consulta del historial. El Core bancario y el sistema de notificaciones son sistemas existentes del banco: para el proyecto, el Core se simula sobre MongoDB Atlas y queda fuera del alcance del producto.

1.4 Organización del repositorio y del documento

Todo el trabajo está versionado en un repositorio de GitHub. La Tabla 1.1 resume su estructura.

Tabla 1.1: *Organización del repositorio*

Carpeta	Contenido
<code>contracts/</code>	Contrato OpenAPI 3.1 (fuente de verdad de la API)
<code>backend/</code>	Implementación en Node.js, pruebas, scripts de carga y despliegue
<code>frontend/</code>	Aplicación web en Next.js
<code>bdd/</code>	Modelo del Core bancario simulado (MongoDB)
<code>docs/informes/</code>	Informes de cada fase en Markdown
<code>docs/diagramas/</code>	Modelo C4 (Structurizr) y vista ArchiMate
<code>docs/evidencias/</code>	Evidencias de pruebas, seguridad, despliegue y carga
<code>docs/latex/</code>	Fuente de este informe
<code>entregables/</code>	Documentos finales para la entrega
<code>.github/workflows/</code>	Pipeline de CI/CD

Los capítulos 2 a 5 corresponden a las cuatro fases del proyecto. El capítulo 6 describe la integración con el frontend y el capítulo 7 presenta las conclusiones. Los anexos reúnen el detalle de los patrones de diseño y las evidencias.

CAPÍTULO 2

Fase 1: justificación de negocio

Supuestos del equipo, no datos verificados: el producto lo lanza un banco ya establecido en Ecuador (existen core bancario y notificaciones), la moneda es USD, y el segmento objetivo son adultos jóvenes que ya tienen cuenta pero no tienen hábito de ahorro. Las cifras de \$5 son ilustrativas. Las afirmaciones de mercado de \$1 están planteadas en términos cualitativos a propósito: antes de entregar hay que respaldarlas con fuentes citables (BCE, Superintendencia de Bancos, Global Findex), porque un dato sin fuente resta más de lo que suma.

2.1 El problema

Del lado del cliente, el ahorro no es un hábito sino un residuo: se ahorra “lo que sobra” a fin de mes, y no sobra. A eso se suma que el retorno es opaco —nadie sabe cuánto va a tener en 18 meses— y que los productos tradicionales piden montos mínimos altos, plazos rígidos y, muchas veces, ir a una agencia.

Del lado del banco, el depósito a la vista es barato pero volátil: puede retirarse cualquier día, lo que encarece los requerimientos de liquidez y limita la colocación a plazo. Captar un ahorrador en agencia cuesta tiempo de personal y papel. Y cuando alguien abandona la apertura en el canal digital, el banco no sabe en qué paso se fue ni por qué.

El cruce de ambos problemas define el producto: automatizar el aporte resuelve el hábito, mostrar la proyección desde el primer clic resuelve la opacidad, y **no exigir monto mínimo** resuelve la barrera. A cambio, el banco obtiene permanencia, recurrencia y un embudo de apertura medible.

Una corrección al tablero: la nota “*activos inmediatos*” está mal nombrada. Un depósito de ahorro es un **pasivo** en el balance. El valor está en que es un pasivo barato y estable que fondea la cartera de crédito, que sí es el activo rentable. Conviene decirlo como “**captación inmediata de depósitos: fondeo estable y de bajo costo**”.

2.2 Qué hace el producto

Billetera de Ahorro permite fijar una meta, elegir cuánto y cuándo aportar, y ver desde el primer minuto cuánto dinero se tendrá al final. El aporte se debita solo; el sistema acompaña con progreso visible, notificaciones y reportes; y si el cliente bloquea su ahorro, gana mejor tasa.

Features

- Simula el retorno en el tiempo con intereses.

- Genera plan de ahorro.
- A través de un stepper el usuario elige objetivo, monto, interés, tiempo y confirmación.
- Ahorro **sin monto mínimo ni máximo**.
- Da opción de bloquear el ahorro para ganar más interés.
- Permite crear varios planes de ahorro.
- Débito automático desde cuenta, con fecha de débito elegible.
- Notificaciones del sistema: débitos, cambios en el plan, fechas.
- Visualización del progreso de la meta: barra de avance y porcentaje de cumplimiento.
- Reportes con el historial de planes y los movimientos de cada uno.
- Retirar parte del saldo cuando el plan no está bloqueado.
- Desbloquear el plan antes de tiempo o cancelarlo y recuperar los fondos; si estaba bloqueado, en ambos casos pierde los intereses devengados.

Beneficios para el banco. Captación inmediata de depósitos; asegura el dinero en ahorro por más tiempo; provee educación bancaria al usuario; ayuda a disminuir las objeciones; barrera de entrada baja para captar un segmento joven; permite levantar datos de dónde se detienen los usuarios al generar el plan; y usa la barra de progreso para sostener la constancia sin costo de canal.

Beneficios para el cliente. Puede ganar más intereses; tiene un objetivo planificado desde el inicio; puede llevar varios objetivos a la vez; ve exactamente cuánto tendrá al final del período; puede iniciar con poco y crecer sin límite; y el interés varía según el valor meta y el tiempo del plan.

Cobranza. El cargo se intenta el día que el cliente eligió y, si falla, se reintenta cada 24 horas hasta **5 veces**. Si al quinto intento no se logró el descuento, **la cuota se cancela y el plazo del plan crece un mes**: no hay mora, no se acumulan deudas y nunca se cobran dos cuotas juntas. El plan se completa cuando el saldo alcanza el objetivo, y recién ahí se liquidan los fondos al cliente. Cada plan genera **su propio cargo**: quien tenga tres planes que se debitan el mismo día ve tres cargos, no uno agregado.

Lo que esta regla cambia es la naturaleza de la promesa: el producto deja de comprometer una **fecha** y compromete un **monto**. El cliente nunca ve un cargo que no reconoce ni entra en mora, pero su meta puede tardar más de lo previsto.

2.2.1 Principios de producto

1. **La proyección antes del compromiso.** El usuario ve cuánto tendrá al final *antes* de registrarse: la simulación es pública y anónima a propósito.
2. **El ahorro no se pide, se programa.** El débito automático es el corazón del producto, no una opción secundaria.
3. **Cada centavo es reconstruible.** El saldo se calcula desde un historial inmutable de movimientos, no se sobrescribe.

2.3 Objetivos de las APIs

Las cinco dimensiones del diseño de APIs (U1 §1.2) aplicadas a este producto:

Dimensión	Cómo se materializa aquí
Interoperabilidad	La capacidad de ahorro se expone como contrato REST consumible por la banca web, la PWA y mañana por un socio externo, sin tocar el core bancario
Modularidad	Frontend y backend evolucionan por separado: la frontera es el <code>openapi.yaml</code> . La tabla de tarifas cambia sin redespargar la app
Escalabilidad	Nuevos tipos de plan se agregan como recursos nuevos, sin refactorizar el núcleo
Estandarización	REST + OpenAPI 3.1 + JWT + JSON. Ningún consumidor necesita documentación privada ni SDK propietario
Seguridad	JWT firmados de corta duración emitidos por un servicio de autenticación interno, refresh token, ledger inmutable y auditoría por movimiento

Los objetivos de negocio que el MVP debe poder medir: captar depósito recurrente (saldo bajo gestión y aportes/mes), aumentar la permanencia (% de planes con bloqueo activo), reducir el costo de adquisición (% de aperturas sin intervención humana), y cerrar el embudo de apertura (conversión de simulación a plan creado).

2.4 La cadena de valor de las APIs

Ubicación en los tres niveles de madurez de U3 §3.2:

Nivel	En este producto	Estado
Supervivencia — conectividad de sistemas	La API integra core bancario, cobros y notificaciones, que hoy son silos	✓ MVP
Ventaja competitiva — habilitación omnicanal	El mismo contrato sirve a web y móvil sin duplicar lógica	✓ MVP
Ventaja competitiva — socios y monetización	Ahorro embebido: un comercio o fintech ofrece “ahorrá para esto” usando nuestra API	(próximo) Fase 2
Diferenciación — máximo valor de negocio	El historial de cumplimiento se vuelve insumo de scoring crediticio propio	(próximo) Visión

El último nivel es lo que hace que valga la pena construirlo como APIs y no como una pantalla más: un cliente que sostuvo 18 aportes es un sujeto de crédito con evidencia, no con estimación, y ese dato no lo tiene ningún buró.

2.5 Monetización

Según la taxonomía de U1 §1.6, este producto es **monetización indirecta**: la API no genera cobro directo, pero impulsa el producto que sí deja margen. No hay pay-per-call, ni suscripción, ni freemium. El usuario no paga por ahorrar y el banco no se cobra a sí mismo por llamar a su propio endpoint.

Las palancas de ingreso son cuatro:

1. **Spread de intermediación.** El banco paga tasa pasiva por el depósito y coloca esos fondos a tasa activa. El margen es la diferencia, por saldo y por permanencia. Es la palanca de mayor volumen.
2. **Calidad del fondeo.** Un depósito bloqueado a plazo es fondeo estable: mejora los indicadores de liquidez y permite colocar a plazos más rentables. Vale más que el mismo monto a la vista, y por eso el bloqueo se paga con mejor tasa.
3. **Reducción del costo de adquisición.** Apertura autoservicio en lugar de agencia. No es ingreso, es margen liberado, y es la palanca de efecto más inmediato.
4. **Canal de socios (futuro).** Ahorro embebido bajo acuerdo B2B. Es el único escenario donde aparecería monetización directa: revenue share o tarifa por volumen.

Ejemplo ilustrativo, no proyección (*reemplazar con las tasas reales del banco*): con cuota promedio de USD 50, permanencia de 18 meses y saldo promedio de $\approx$ USD 450, un spread de 10 puntos entre tasa pasiva y activa deja un margen financiero bruto de $\approx$ **USD 67 por cliente en el ciclo**. A eso se le resta el costo de servicio del canal digital.

La conclusión importa más que el número: el margen unitario es pequeño, **el negocio es de volumen y permanencia**. Por eso la ausencia de monto mínimo prioriza la conversión de simulaciones en planes, el bloqueo es la feature con mejor retorno por esfuerzo, y la arquitectura tiene que ser barata de operar —un costo de servicio alto se come el margen.

Lo que se descartó y por qué. *Pay-per-call*: el consumidor es la propia app del banco; cobrarse a sí mismo es contabilidad interna, no ingreso. *Freemium con límite de planes*: contradice la feature de múltiples planes y reduce el depósito captado. *Comisión al ahorrador*: cobrar por ahorrar destruye la propuesta de valor. *Venta de datos*: inaceptable legal y éticamente; el dato se usa internamente para scoring propio, con base legal y consentimiento.

2.6 Ecosistema de APIs

Siguiendo la clasificación de U1 §1.5, el MVP es principalmente de **ecosistema interno**: las APIs sirven a la app propia y quedan disponibles para otros equipos del banco. Hay una porción **pública acotada** —solo la simulación y la consulta de tarifas son anónimas, para permitir simular antes de registrarse— y un **ecosistema de socios** previsto para Fase 2, bajo contrato formal y credenciales JWT con permisos restringidos.

Exponer la simulación sin autenticación es una decisión de negocio (bajar la barrera de entrada) con consecuencia arquitectónica: es superficie de ataque y de abuso de costo, y por eso lleva rate limiting más estricto que los endpoints autenticados.

2.7 Anatomía de la API

Las 8 capas de U1 §1.3 aplicadas a esta API:

#	Capa	En Billetera de Ahorro
1	Endpoints	/v1/auth/token · /v1/tarifas · /v1/simulaciones · /v1/cuentas-debito · /v1/planes-ahorro · /v1/clientes/me · /v1/planes-ahorro/{planId} y sus sub-recursos /bloqueo, /desbloqueo, /cancelacion, /aportes, /retiros, /cuotas, /movimientos · /v1/corridas-cobro. Las notificaciones no tienen endpoint: las envía el sistema de notificaciones existente al consumir eventos
2	Métodos	GET consultar y simular (sin efectos secundarios, cacheable) · POST crear, bloquear, desbloquear, cancelar, aportar, retirar · PATCH cambiar datos descriptivos o fecha de débito. Sin DELETE: un plan se cancela como cambio de estado; el historial financiero no se borra
3	Recursos	Cliente, PlanAhorro, Simulacion, Tarifa, Cuota, Aporte, Retiro, Movimiento, CuentaDebito, CorridaCobro
4	Parámetros	<i>Path:</i> {planId} · <i>Query:</i> ?estado=, ?pagina=, ?cursor=, ?limite= · <i>Headers:</i> Authorization, Idempotency-Key, If-Match, X-Request-Id · <i>Body:</i> JSON validado contra el esquema
5	Formato	JSON. Montos como entero de centavos + ISO 4217. Fechas ISO 8601 UTC. Errores en application/problem+json

#	Capa	En Billetera de Ahorro
6	Códigos de estado	200 · 201+Location · 202 (cobro aceptado, asíncrono) · 400 · 401 · 403 · 404 · 409 (conflicto de estado) · 412 (versión desactualizada) · 429+Retry-After · 500 · 503 (Core no disponible)
7	Autenticación / Autorización	Servicio de autenticación interno que emite JWT (RS256) de corta duración + refresh token; roles y permisos en los claims; RBAC (CLIENTE, OPERADOR, AUDITOR) con scopes por recurso (perfil:leer, planes:leer, planes:escribir, aportes:escribir, retiros:escribir, movimientos:leer, cuentas:leer, corridas:leer); ABAC para titularidad del plan
8	Documentación	contracts/openapi.yaml (OpenAPI 3.1) como única fuente de verdad, publicado con Swagger UI

Un ejemplo del viaje completo: el cliente envía POST /v1/planes-ahorro (1) con verbo de creación (2), el cuerpo con meta, fecha objetivo, día de débito y cuenta (3), header **Authorization: Bearer <JWT>** que el gateway valida (7) e **Idempotency-Key** para que un reintento de red no cree dos planes. El servicio valida la cuenta con el Core, deriva el plazo de la fecha objetivo, calcula la cuota, congela la tarifa y genera el calendario de cuotas. Responde 201 **Created** (6) con Location y un JSON (5) que incluye el estado del plan y la proyección del monto final.

2.8 Naturaleza de la API

Las cuatro dimensiones del contrato digital (U1 §1.4):

Naturaleza	Cómo se manifiesta
Intermediaria	Es el puente entre el cliente y dos sistemas que él nunca ve: el core bancario y el motor de notificaciones. El cliente pide “crear mi plan”; la API traduce eso a conversaciones distintas

Naturaleza	Cómo se manifiesta
Abstracta	El consumidor no sabe —ni necesita saber— que el saldo se reconstruye desde un ledger de eventos, que el cobro es un proceso batch con checkpoint, ni que hay un circuit breaker protegiendo la llamada al core. Pide un plan y recibe un saldo
Contractual	El <code>openapi.yaml</code> define entradas y salidas antes de que exista código. El contrato es doble: el técnico con el desarrollador y el financiero con el cliente, porque la tasa aplicada queda fijada en el plan al contratar
Evolutiva	Versionamiento SemVer: los cambios aditivos no rompen integraciones; uno disruptivo exige salto de versión mayor, documentación paralela de v1 y v2 y guía de migración

Sobre el ciclo de vida (U1 §1.8): hoy estamos en **diseño**, estableciendo el contrato **v1**. La **evolución** traerá el canal de socios con v1 y v2 conviviendo. El **retiro** de v1 se anunciará con ventana de migración y responderá **410 Gone** en los endpoints dados de baja.

2.9 Indicadores

Categoría	Qué se mide
Negocio	Saldo bajo gestión · aportes recurrentes/mes · % de planes bloqueados · % de cuotas canceladas por falta de fondos
Producto	Conversión simulación → plan creado · planes activos por cliente
API	p95 < 500 ms · error rate < 1 % · RPS sostenido · punto de ruptura (validados con k6 en Fase 4)
Operación	% de cobros exitosos al primer intento · corridas de cobro completadas sin reinicio

La conversión y el abandono **por paso del stepper** quedan como trabajo futuro: el backend solo ve la simulación y la creación del plan, así que el MVP mide el embudo entre esos dos puntos.

2.10 Riesgos

Riesgo	Mitigación
El plazo se estira sin tope si el cliente falla cuotas repetidamente, y la meta se aleja	Notificar cada prórroga con la nueva fecha estimada y sugerir bajar la cuota antes que abandonar el plan
Cuotas elegidas por optimismo que el cliente no logra sostener	La simulación muestra la proyección exacta antes de comprometerse, y la fecha de débito elegible permite alinear el cargo con el día de cobro
Los retiros parciales reducen la permanencia , que es de donde sale el margen (§5)	Un plan bloqueado no admite retiros, y salir del bloqueo cuesta los intereses devengados: el incentivo a mantener el dinero está en el bloqueo, no en prohibir el retiro
Sin monto mínimo, una cuota muy pequeña cuesta más cobrarla que lo que capta	Aceptado para el alcance del proyecto porque refuerza la barrera de entrada cero; un producto real necesitaría un piso operativo

2.11 Qué queda por decidir

1. ¿Cuándo se suspende un plan cuyo plazo se estira indefinidamente? *(La otra mitad de esta pregunta quedó resuelta en la Fase 2, §4.2: un error técnico del banco no consume ninguno de los 5 intentos.)*
2. Reemplazar las cifras ilustrativas de §5 por tasas reales o un rango citado.
3. Respaldar las afirmaciones de mercado de §1 con fuentes citables.

2.12 Requerimientos del producto

2.12.1 Requerimientos funcionales

ID	Requerimiento	Nota
RF-01.1	Crear uno o varios planes con nombre, monto objetivo y fecha objetivo	Sin monto mínimo ni máximo; el plazo se deriva de la fecha y la cuota la calcula el sistema
RF-01.2	Modalidad de ahorro: cuota fija mensual	
RF-01.3	Elegir y modificar la fecha mensual del débito	
RF-01.4	Consultar y filtrar el historial de planes (activos, completados, cancelados)	Filtros + paginación por offset
RF-01.5	Consultar los movimientos de un plan	Paginación por cursor

ID	Requerimiento	Nota
RF-01.6	Bloquear el plan a plazo para obtener mejor tasa	Un booleano y una tasa; sostiene la monetización
RF-01.7	Desbloquear el plan antes de tiempo	La tasa vuelve a la base y se pierden los intereses devengados (misma regla que RF-04.2)
RF-02.1	Simular el rendimiento proyectado (monto acumulado + intereses)	Público y anónimo
RF-03.1	Programar y ejecutar el débito automático	Política de 5 intentos a 24 h
RF-04.1	Cancelar un plan y devolver los fondos a la cuenta principal	
RF-04.2	Penalidad al cancelar un plan bloqueado: pierde los intereses devengados	Regla única, sin tabla
RF-04.3	Retirar parte del saldo de un plan no bloqueado a una cuenta del cliente	Hasta el saldo disponible; un plan bloqueado no admite retiros
RF-05.1	Mostrar el progreso de la meta (barra y porcentaje)	Campo calculado saldo / montoMeta , no es un endpoint
RF-05.2	Notificar débitos ejecutados y eventos del plan	Demuestra EDA

2.12.2 Requerimientos no funcionales

ID	Requerimiento	Nota
RNF-01.1	p95 < 500 ms en endpoints síncronos	Umbral exigido por la rúbrica
RNF-01.2	Procesamiento batch del corte de débitos	
RNF-01.3	Escalado horizontal automático	
RNF-02.1	JWT de corta duración emitido por un servicio de autenticación interno	Exigido por la rúbrica
RNF-02.2	TLS 1.3 en tránsito, AES-256 en reposo	
RNF-02.3	Consistencia ACID en los movimientos de saldo	
RNF-03.1	Desacoplamiento del core legacy vía EDA	

ID	Requerimiento	Nota
RNF-03.2	Disponibilidad 99.9 %	Meta de diseño, no resultado medido
RNF-03.3	Auditoría inmutable por transacción	La provee el ledger

CAPÍTULO 3

Fase 2: arquitectura y patrones

3.1 Introducción

3.1.1 Propósito del documento

En la Fase 1 justificamos el caso de negocio de la Billetera de Ahorro. En esta segunda fase definimos cómo se va a construir: elegimos el estilo arquitectónico, armamos los diagramas de la solución y justificamos los patrones de diseño que decidimos aplicar.

Para tomar las decisiones usamos las herramientas que revisamos en la Unidad 2 de la asignatura (Prieto, 2026): la matriz de calidad ISO/IEC 25010, el árbol de decisiones de interacción de datos, los tres pilares de patrones y la matriz de diagnóstico.

Los patrones se presentan en dos niveles. En el nivel de arquitectura (sección 4.1) están los patrones orientados a APIs que organizan los contenedores: API Gateway, Circuit Breaker, paginación, publicación/suscripción, entre otros. En el nivel de código (sección 4.5) están los patrones de diseño clásicos (Gamma et al., 1994) con los que cada contenedor implementa esas decisiones.

3.1.2 Requisitos que guían la arquitectura

Al revisar los requisitos de la Fase 1 notamos que no todos influyen igual en la arquitectura. La siguiente tabla recoge los que más pesaron en nuestras decisiones:

Tabla 3.1: *Requisitos con impacto en la arquitectura*

Requisito	Descripción	Impacto en la arquitectura
RNF-01.1	p95 menor a 500 ms en endpoints síncronos	Caché HTTP, paginación y pocas llamadas síncronas
RNF-01.2	Procesamiento batch del corte de débitos	Batch Processor y uso de eventos para absorber el pico
RNF-02.1	JWT de corta duración emitido por un servicio interno	Servicio de Autenticación y validación en el Gateway
RNF-02.3	Consistencia ACID en los movimientos de saldo	Una sola base relacional para las escrituras y Transactional Outbox para publicar eventos
RNF-03.1	Desacoplamiento del Core legacy vía EDA	Message Broker y Adaptador Core
RNF-03.2	Disponibilidad de 99.9 %	Circuit Breaker y Retry con jitter

Requisito	Descripción	Impacto en la arquitectura
RNF-03.3	Auditoría inmutable por transacción	Ledger de solo inserción
RF-01.4 y RF-01.5	Historial de planes y de movimientos	Paginación por offset y por cursor, filtros
RF-02.1	Simulación pública sin autenticación	Rate Limiting y caché
RF-03.1	Débito automático con 5 intentos cada 24 h	Reintento de negocio a cargo del Batch, modelado con el patrón State
RF-04.2 y RF-01.7	Penalidad al cancelar o desbloquear un plan bloqueado	Política de salida del bloqueo intercambiable (patrón Strategy)
RF-04.3	Retiro parcial a una cuenta del cliente	Evento CreditoSolicitado, simétrico al débito

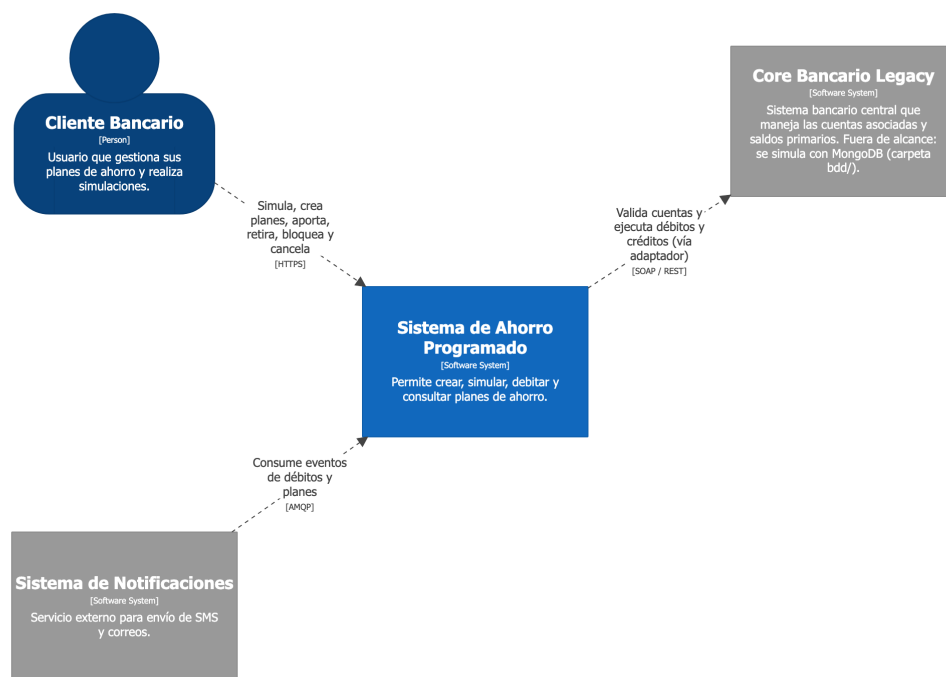
3.2 Arquitectura de software

3.2.1 Capas de la arquitectura

Organizamos los componentes según las cuatro capas de una arquitectura orientada a APIs vistas en clase. En la capa de integración ubicamos todo lo que protege al dominio de lo externo: el Gateway frente a los clientes y el Adaptador frente al Core legacy.

Tabla 3.2: *Capas y componentes de la solución*

Capa	Componentes	Responsabilidad
Presentación	Banca web (SPA en Next.js)	Interfaz para simular, crear, consultar, aportar, retirar, bloquear, desbloquear y cancelar planes
Integración	API Gateway (Kong), Adaptador Core y Message Broker (RabbitMQ)	Punto de entrada, seguridad perimetral, traducción hacia el legacy y mensajería asíncrona
Aplicación	Ahorro Core API, Servicio de Autenticación y Batch Processor	Reglas de negocio de los planes, emisión de tokens y corte de débitos
Acceso a datos	PostgreSQL 15	Planes, calendario de aportes, usuarios, ledger y tabla outbox, con transacciones ACID



System Context View: Sistema de Ahorro Programado
Wednesday, September 30, 2026 at 9:59 PM Ecuador Time

Figura 3.1: Diagrama de contexto (C4, nivel 1)

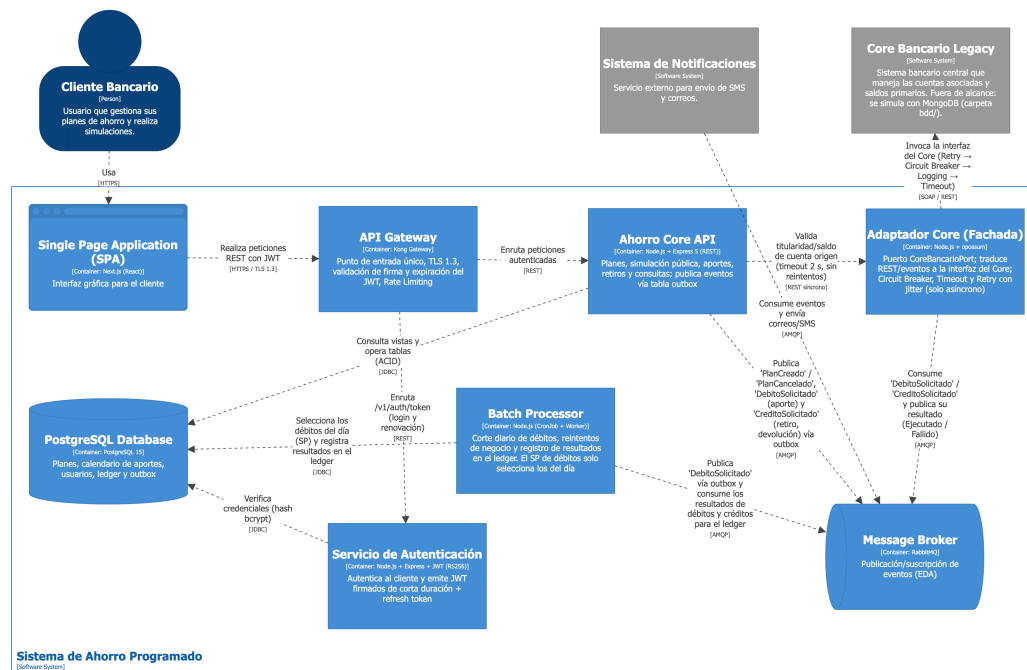
3.2.2 Diagrama de contexto

Modelamos la arquitectura con el enfoque C4 en Structurizr DSL (archivo docs/diagramas/workspace.dsl). A nivel de contexto el sistema solo depende de dos sistemas que el banco ya tiene: el Core bancario, que sigue siendo el responsable de las cuentas y de las operaciones contables, y el sistema de notificaciones.

Ambos sistemas quedan fuera del alcance del proyecto. Para poder ejecutar el flujo completo, el Core bancario se simula con una base MongoDB Atlas (modelo de la carpeta bdd/) que reproduce clientes, cuentas y saldos como lo haría un core de producción. La Billetera de Ahorro no accede a esa base directamente: toda interacción pasa por el Adaptador Core, igual que ocurriría con el Core real.

3.2.3 Diagrama de contenedores

Decidimos desplegar la solución como un monolito modular y no como microservicios. Todo el backend es un solo proyecto en Node.js (TypeScript) organizado en capas (dominio, aplicación, infraestructura y presentación), con un punto de entrada por proceso. En la infografía de rediseño vista en clase, esta opción aparece como la adecuada para equipos pequeños y dominios simples, y ese es nuestro caso: tenemos un solo dominio (los planes de ahorro) y un equipo de cinco personas. Separamos en contenedores propios únicamente el Batch Processor, el Adaptador Core y el Servicio de Autenticación, porque tienen un



Container View: Sistema de Ahorro Programado
Wednesday, September 30, 2026 at 9:59 PM Ecuador Time

Figura 3.2: Diagrama de contenedores (C4, nivel 2)

ciclo de ejecución o requisitos de seguridad distintos a los de la API.

La Ahorro Core API y el Batch Processor no publican directamente en RabbitMQ: guardan el evento en una tabla outbox dentro de la misma transacción que la operación que lo origina, y un publicador lo envía al broker después del commit (sección 4.1).

Tabla 3.3: Contenedores del sistema

Contenedor	Tecnología	Responsabilidad
SPA	Next.js (React)	Interfaz del cliente
API Gateway	Kong Gateway	Punto de entrada único, TLS 1.3, validación de firma y expiración del JWT, Rate Limiting
Servicio de Autenticación	Node.js + Express + JWT (RS256)	Verifica credenciales y emite el access token y el refresh token
Ahorro Core API	Node.js + Express 5 (REST)	Planes, simulación pública, aportes bajo solicitud y consultas; publica eventos mediante la tabla outbox
Batch Processor	Node.js (CronJob + Worker)	Corte diario de débitos automáticos, reintentos de negocio y registro en el ledger del resultado de todos los débitos

Contenedor	Tecnología	Responsabilidad
Adaptador Core	Node.js + opossum (Circuit Breaker)	Único punto de contacto con el Core legacy, detrás del puerto CoreBancarioPort
Message Broker	RabbitMQ	Publicación y suscripción de eventos
Base de datos	PostgreSQL 15	Datos del dominio, ledger y tabla outbox

3.2.4 Vista ArchiMate

Actualizamos la vista ArchiMate de la Fase 1 para que coincida con los contenedores anteriores. En la capa de motivación agregamos un segundo requisito de débito, porque el producto maneja dos casos: el débito automático, que ejecuta el Batch Processor en la fecha programada, y el aporte que el cliente solicita desde la aplicación. Los dos terminan publicando el mismo evento DebitoSolicitado, así que comparten el camino hacia el Core, la resiliencia y la trazabilidad.

3.3 Selección del estilo arquitectónico

3.3.1 Evaluación con ISO/IEC 25010

Como se vio en clase, no hay un estilo que sea mejor en todo: cada uno gana en unos atributos de calidad y pierde en otros. Por eso comparamos los cuatro estilos candidatos con la matriz de la Unidad 2 y agregamos una fila con nuestra valoración para este caso.

Tabla 3.4: Comparación de estilos según ISO/IEC 25010

Atributo	REST	GraphQL	gRPC	EDA
Facilidad de cacheo	Alto	Bajo	Bajo	N/A
Escalabilidad concurrente	Medio	Medio	Alto	Alto
Mantenibilidad	Alto	Medio	Medio	Bajo
Seguridad perimetral	Medio	Bajo	Alto	Medio
¿Aplica a nuestro caso?	Sí. Cliente web, recursos simples y simulación que se puede cachear	No. No hay datos jerárquicos ni varios clientes, y se pierde el cacheo	No. El navegador no lo soporta de forma nativa y hay un solo servicio	Sí. Hay un pico diario de débitos y un Core legacy

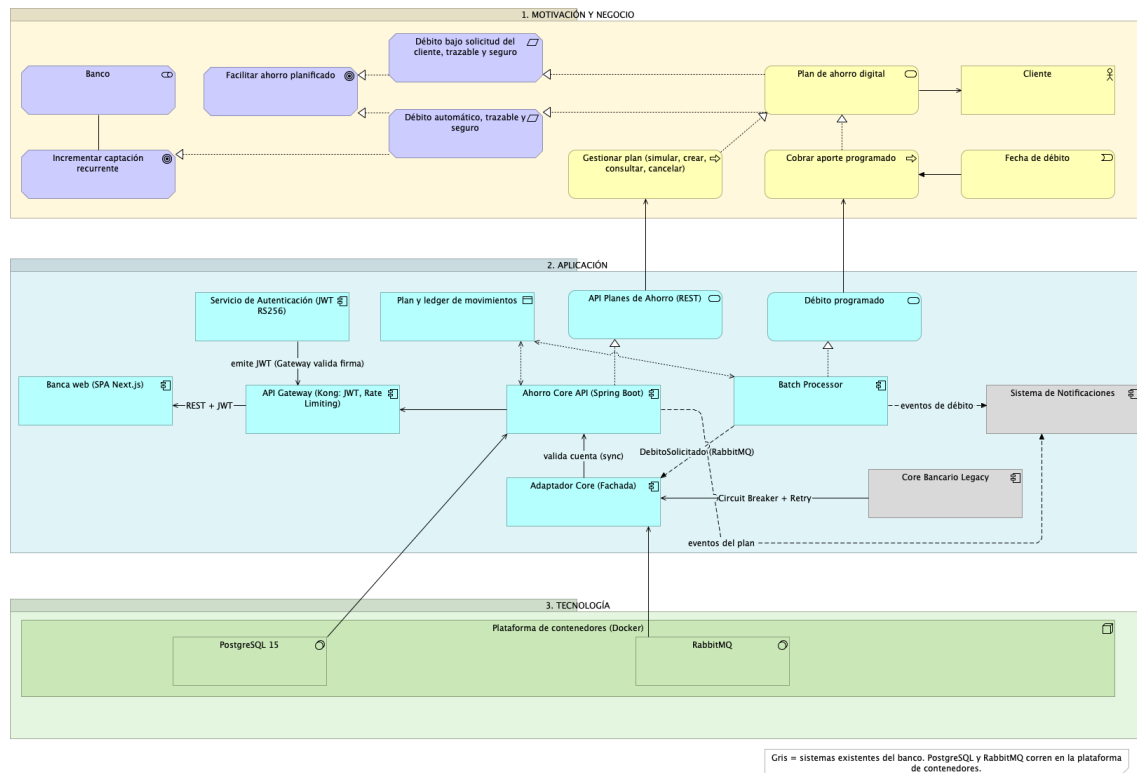


Figura 3.3: Vista ArchiMate de la Fase 2

3.3.2 Árbol de decisiones

Recorrimos el árbol de decisiones de la clase con las características del proyecto. La primera pregunta es si hay comunicación interna de alta densidad entre microservicios; en nuestro caso no, porque existe un solo servicio de dominio, así que gRPC no aporta. La segunda pregunta es si hay picos masivos que puedan saturar la base de datos, y aquí la respuesta es sí: el corte diario concentra miles de débitos en pocas horas, lo que nos llevó a desacoplar ese flujo con EDA. Por último, la interfaz no maneja datos jerárquicos complejos, ya que planes y movimientos son recursos planos, por lo que para el consumo del cliente nos quedamos con REST y OpenAPI.

3.3.3 Decisión

Con base en lo anterior, decidimos usar un estilo híbrido. REST (OpenAPI 3.1 y JSON) atiende las operaciones síncronas del cliente, y EDA con RabbitMQ se encarga de los débitos, las notificaciones y la integración con el Core.

Esta decisión tiene un costo que aceptamos de forma consciente. Uno de los principios vistos en clase dice que la complejidad no se destruye, se transfiere: EDA nos da aislamiento temporal frente al legacy, pero a cambio tenemos consistencia eventual y una trazabilidad más difícil. Para mitigarlo, cada evento lleva un correlation-id y la API expone el estado del débito (PENDIENTE, EJECUTADO o FALLIDO), de modo que el cliente no depende de una respuesta inmediata del Core.

3.4 Patrones de diseño

3.4.1 Patrones aplicados

La Tabla 5 resume los patrones que aplicamos. Para cada uno indicamos el pilar al que pertenece según la clasificación de la Unidad 2 y el problema que resuelve; cuando corresponde, usamos el síntoma tal como aparece en la matriz de diagnóstico.

Tabla 3.5: *Patrones aplicados*

Patrón	Pilar	Dónde se aplica	Problema que resuelve
API Gateway	Borde	Kong, como entrada única	Centraliza TLS, validación del JWT, enrutamiento y cuotas (RNF-02.1)
Rate Limiting	Resiliencia	Kong, con un límite menor para anónimos en /simulaciones	Abuso del endpoint público; se responde 429 con Retry-After (RF-02.1)
Fachada / Adaptador	Integración	Adaptador Core	Aísla el dominio del protocolo legacy; si cambia el Core, solo se modifica el adaptador (RNF-03.1)
Circuit Breaker	Resiliencia	Llamadas del Adaptador al Core legacy	Cascadas de error por validaciones lentas; protege el pool de hilos (RNF-03.2)
Retry con backoff exponencial y jitter	Resiliencia	Llamadas asíncronas al Core (consumo de DebitoSolicitado y CreditoSolicitado): 1 s, 2 s y 4 s más un valor aleatorio	Caídas por reintentos simultáneos (efecto estampida)
Timeout	Resiliencia	Llamada síncrona de validación de cuenta: 2 s, sin reintentos con espera	Que una respuesta lenta del Core no rompa el p95 de la API (RNF-01.1)
Paginación y filtrado	Consumo	GET /v1/planes-ahorro (offset) y GET /v1/planes-ahorro/{planId}/movimientos (cursor)	Respuestas grandes y desplazamiento del offset (RF-01.4 y RF-01.5)

Patrón	Pilar	Dónde se aplica	Problema que resuelve
Caché HTTP	Consumo	GET /v1/simulaciones con Cache-Control	La misma entrada da el mismo resultado; baja la latencia (RNF-01.1)
Publicación / suscripción	Estilo EDA	La API y el Batch publican; el Adaptador, el Batch y Notificaciones consumen	Acoplamiento temporal con el Core y pico del corte (RNF-03.1 y RF-05.2)
Transactional Outbox	Integración	Tabla outbox en PostgreSQL, en la API y el Batch	Que nunca quede un cambio guardado sin su evento ni un evento publicado de un cambio que no se guardó (RNF-02.3)
Consumidor idempotente	Resiliencia	Todos los consumidores de RabbitMQ, usando el id del evento	El broker puede entregar un mensaje más de una vez; un duplicado no debe generar un segundo débito ni un segundo asiento
Dead Letter Queue	Resiliencia	Cada cola de RabbitMQ tiene su cola <code>.dlq</code>	Un mensaje que falla dos veces no se pierde: queda apartado para revisarlo y reprocesarlo
Ledger de solo inserción	Persistencia	Tabla ledger en PostgreSQL	Auditoría inmutable; el saldo se obtiene sumando movimientos (RNF-03.3)
Idempotency-Key	Resiliencia	POST /v1/planes-ahorro y los POST de /aportes, /retiros, /desbloqueo y /cancelacion	Evita que un reintento del cliente cree un plan duplicado o mueva dinero dos veces

3.4.2 Resiliencia frente al Core legacy

El Circuit Breaker del Adaptador sigue la máquina de estados que se estudió en clase. Si más del 50 % de las llamadas fallan en una ventana de 10 segundos, pasa de cerrado a abierto y deja de llamar al Core; mientras está abierto responde de inmediato con un fallo controlado (un 503 para el cliente o un evento DebitoFallido con causa técnica). Después de 30 segundos pasa a semiabierto y deja pasar algunas llamadas de prueba antes de volver a cerrarse.

Durante el análisis nos dimos cuenta de que había que diferenciar dos tipos de reintento. El reintento técnico (Retry con jitter) repite en cuestión de segundos una llamada que falló por red o por timeout. El reintento de negocio (RF-03.1), en cambio, lo programa el Batch cada 24 horas, hasta cinco veces, cuando el Core rechaza el cargo. Con esta separación respondimos una pregunta que había quedado abierta en el tablero de ideación: un error técnico del banco no le consume al cliente ninguno de sus cinco intentos.

También distinguimos los dos caminos que llegan al Core, porque no toleran la misma espera:

Tabla 3.6: *Resiliencia según el camino hacia el Core*

Camino	Quién espera	Resiliencia aplicada
Síncrono: validar titularidad y saldo de la cuenta origen	El cliente, en línea	Timeout de 2 s y Circuit Breaker, sin reintentos con espera. Si falla, la API responde 503 y el cliente puede volver a intentar
Asíncrono: ejecutar el débito o el crédito al consumir DebitoSolicitado o CreditoSolicitado	Nadie en línea	Circuit Breaker y Retry con backoff de 1 s, 2 s y 4 s más jitter. Si se agotan, se publica DebitoFallido o CreditoFallido con causa técnica

Aplicar el backoff completo en el camino síncrono habría hecho que una sola validación fallida tardara más de 7 segundos, incompatible con el p95 menor a 500 ms (RNF-01.1).

Ambas políticas se componen con decoradores propios (el Circuit Breaker usa la librería opossum) en el orden Retry → Circuit Breaker → Logging → Timeout → adaptador. El Retry va por fuera para que cada intento cuente como una llamada en la ventana del Circuit Breaker; cuando el circuito está abierto, el Retry no reintenta y el fallo se devuelve de inmediato.

3.4.3 Paginación y filtrado

Usamos las dos estrategias de paginación vistas en la Unidad 3, cada una donde encaja mejor. Para los planes elegimos offset, porque cada cliente tiene pocos y así puede saltar a una página específica. Para los movimientos elegimos cursor, porque crecen sin límite y los registros nuevos del ledger desplazarían los resultados si usáramos offset.

```
GET /v1/planes-ahorro?estado=ACTIVO&pagina=1&limite=10
GET /v1/planes-ahorro/{planId}/movimientos?cursor=eyJpZCI6MTIwfQ&limite=20

{ "items": [ { "id": 120, "tipo": "APORTE_AUTOMATICO", "montoCentavos": 9746, "moneda": "USD" } ],
  "siguienteCursor": "eyJpZCI6MTAwfQ", "hayMas": true }
```

Siguiendo lo definido en la anatomía de la API (Fase 1), los montos viajan como enteros en centavos junto con su código de moneda ISO 4217, para evitar errores de redondeo con decimales.

3.4.4 Seguridad en el borde

Optamos por un servicio de autenticación propio basado en JWT. Este servicio verifica las credenciales del cliente (guardadas con hash bcrypt) y emite un access token firmado con RS256, válido por 15 minutos, junto con un refresh token. El Gateway valida la firma y la expiración con la clave pública, sin consultar al servicio en cada petición, lo que ayuda a mantener bajo el tiempo de respuesta. Los roles (CLIENTE, OPERADOR y AUDITOR) viajan en los claims del token y, además, la API verifica que el plan pertenezca al cliente que hace la solicitud.

3.4.5 Patrones de diseño a nivel de código

Los patrones anteriores definen cómo se relacionan los contenedores. Para implementarlos dentro de cada contenedor aplicamos cinco patrones de diseño clásicos (Gamma et al., 1994). El detalle de cada uno está en el documento PATRONES-DISEÑO.md; aquí resumimos dónde se aplican y qué decisión de arquitectura sostienen.

Tabla 3.7: *Patrones de diseño a nivel de código*

Patrón	Dónde	Decisión de arquitectura que implementa
Adapter	Adaptador Core: puerto CoreBancarioPort con dos implementaciones, CoreLegacyAdapter (habla con el Core) y CoreSimuladoAdapter (respuestas en memoria para pruebas unitarias)	Fachada / Adaptador: si el Core cambia, solo cambia el adaptador (RNF-03.1)
Decorator	Adaptador Core: Retry → Circuit Breaker → Logging → Timeout envolviendo al adaptador	Circuit Breaker, Retry y Timeout se agregan por composición, sin modificar el adaptador (RNF-03.2)
Strategy	Ahorro Core API: CalculoInteresStrategy (tasa base o con bono por bloqueo) y PoliticaSalidaBloqueoStrategy (con o sin pérdida de intereses al cancelar o desbloquear)	La simulación pública y el plan real usan el mismo cálculo (RF-02.1, RF-01.6, RF-01.7, RF-04.2)
State	Ahorro Core API (ciclo de vida del plan) y Batch Processor (ciclo de vida del débito)	Las transiciones inválidas se traducen en 409 Conflict; la política de cinco intentos queda en un solo lugar (RF-03.1)

Observer	Eventos de dominio de cada servicio, guardados en la outbox y publicados al broker por un relay	Publicación / suscripción y Transactional Outbox (RNF-03.1, RF-05.2)
----------	---	--

El patrón State modela estas transiciones:

- **Plan:** ACTIVO → COMPLETADO cuando el saldo alcanza la meta, y ACTIVO → CANCELADO a pedido del cliente. COMPLETADO y CANCELADO son estados finales: no admiten aportes, retiros ni cancelación. Dentro de ACTIVO, el bloqueo es un sub-estado: bloqueado ↔ no bloqueado; mientras está bloqueado no admite retiros, y salir del bloqueo descuenta los intereses devengados.
- **Débito:** PENDIENTE → EJECUTADO, o PENDIENTE → FALLIDO cuando el Core rechaza el cargo. Un débito automático fallido se reprograma a las 24 horas hasta cinco intentos; al quinto, la cuota se cancela y el plazo del plan se extiende un mes, como define la regla de cobranza de la Fase 1. Un aporte bajo solicitud no se reprograma: si falla, el cliente ve el estado FALLIDO y decide si lo vuelve a pedir.

La Tabla 8 muestra quién publica y quién consume cada evento. El Batch Processor consume el resultado de todo movimiento de dinero con el Core (débitos y créditos) y lo registra en el ledger, así que el asiento contable se implementa una sola vez:

Tabla 3.8: *Eventos del sistema*

Publica	Evento	Consume
Ahorro Core API	PlanCreado	Sistema de Notificaciones
Ahorro Core API	PlanCancelado	Sistema de Notificaciones
Ahorro Core API (aporte bajo solicitud) y Batch Processor (débito automático)	DebitoSolicitado	Adaptador Core
Ahorro Core API (retiro parcial y devolución al cancelar)	CreditoSolicitado	Adaptador Core
Adaptador Core	DebitoEjecutado, DebitoFallido, CreditoEjecutado, CreditoFallido	Batch Processor (registro en el ledger y reintentos) y Sistema de Notificaciones

Para que State y Strategy apliquen también en el Batch, el procedimiento almacenado `sp_procesar_debitos_ahorro_programado()` se limita a seleccionar los débitos del día. Las transiciones de estado, los reintentos y los cálculos quedan en el código del Batch, donde se pueden cubrir con pruebas unitarias.

3.4.6 Patrones evaluados y descartados

También evaluamos otros patrones del catálogo de la unidad que, por ahora, no se justifican en el proyecto:

Tabla 3.9: *Patrones descartados*

Patrón	Motivo del descarte
BFF	En el MVP hay un solo cliente web. Lo volveríamos a evaluar al incorporar la app móvil o a los socios.
CQRS	El volumen no lo justifica y separar los modelos pondría en riesgo la consistencia ACID del saldo (RNF-02.3). Una vista SQL es suficiente para las lecturas.
Service Registry	Solo hay un servicio de dominio; el DNS de la plataforma de contenedores alcanza.
API Composition	No hay varios servicios internos cuyas respuestas haya que combinar.
Strangler Fig	No reemplazamos funcionalidades del Core; el módulo es nuevo y convive con él a través del adaptador.

3.5 Conclusiones

El estilo híbrido REST y EDA surgió de aplicar el árbol de decisiones a nuestro caso: REST resuelve bien lo que consume el cliente, y EDA nos permite absorber el pico del corte diario sin quedar atados a la disponibilidad del Core legacy.

Procuramos que cada patrón respondiera a un requisito concreto y no aplicar patrones solo por aplicarlos. Así quedaron cubiertos los que la rúbrica considera esenciales (paginación, filtrado y resiliencia) y dejamos documentados los descartados con su motivo, para retomarlos si el producto crece hacia más canales o hacia socios externos.

Un resultado que no esperábamos al inicio fue que los dos tipos de débito pudieran compartir el mismo evento. Gracias a eso, la resiliencia y la auditoría se implementan una sola vez para ambos casos.

Por último, bajar las decisiones al nivel de código con Adapter, Decorator, Strategy, State y Observer nos obligó a precisar dos puntos que el diagrama dejaba abiertos: la resiliencia no puede ser la misma en el camino síncrono que en el asíncrono, y la publicación de eventos necesita una outbox para no romper la consistencia que exige el saldo.

CAPÍTULO 4

Fase 3: modelo de datos y contrato de la API

4.1 Enfoque contract-first

Escribimos el contrato antes que el código. El archivo `contracts/openapi.yaml` (OpenAPI 3.1) es la única fuente de verdad entre el frontend y el backend: los DTOs de validación del backend, los tipos del frontend y las pruebas siguen su forma. El contrato se valida con Redocly en cada ejecución del pipeline y se publica con Swagger UI junto con la API, en `http://3.151.57.252/docs/` (Figura 4.1).

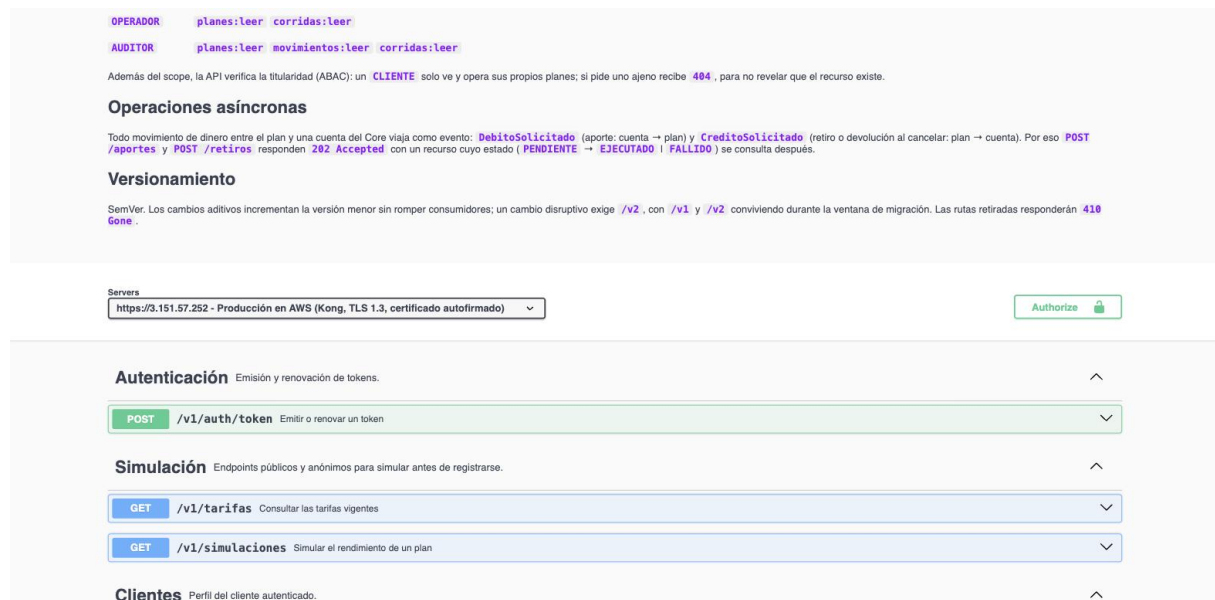


Figura 4.1: Contrato publicado con Swagger UI en el entorno de AWS

4.2 Convenciones del contrato

Tabla 4.1: *Convenciones de diseño del contrato*

Aspecto	Decisión
Rutas	Sustantivos en plural y versión mayor en la ruta (<code>/v1</code>). Las acciones que no son CRUD son sub-recursos (<code>/bloqueo</code> , <code>/desbloqueo</code> , <code>/cancelacion</code> , <code>/aportes</code> , <code>/retiros</code>). No hay DELETE: el historial financiero no se borra.
Montos	Enteros en centavos (<code>*Centavos</code>) junto con el código ISO 4217; nunca decimales para dinero.
Fechas	ISO 8601: <code>date</code> para fechas de calendario y <code>date-time</code> en UTC para instantes.
Errores	<code>application/problem+json</code> (RFC 9457) con un <code>codigo</code> estable para el consumidor.
Paginación	Offset (<code>pagina</code> , <code>limite</code>) en planes, cuotas y corridas; cursor opaco (<code>cursor</code> , <code>limite</code>) en movimientos, que crecen sin límite.
Operaciones asíncronas	Aportes y retiros responden 202 <code>Accepted</code> con un recurso cuyo estado (<code>PENDIENTE</code> , <code>EJECUTADO</code> , <code>FALLIDO</code>) se consulta en <code>Location</code> .
Concurrencia	<code>Idempotency-Key</code> en las operaciones que mueven dinero; <code>ETag/If-Match</code> al modificar un plan (412 si cambió).
Seguridad	Esquema <code>bearerAuth</code> (JWT); cada operación declara los scopes que exige.
Versionamiento	<code>SemVer</code> : los cambios aditivos suben la versión menor; un cambio disruptivo exige <code>/v2</code> .

4.3 Modelo de datos orientado a la API

El modelo que expone la API no copia las tablas de la base de datos: representa lo que el consumidor necesita. Por ejemplo, el saldo del plan no se almacena, se calcula desde el ledger; el progreso es un campo calculado; y la cuenta de débito viaja enmascarada. La Tabla 4.2 resume los recursos.

Tabla 4.2: *Recursos del modelo de la API*

Recurso	Contenido y reglas
PlanAhorro	Meta y fecha objetivo, cuota calculada, plazo y prórrogas, día de débito, cuenta, bloqueo, tasas (TNA base, bono, total y TEA), saldo y saldo disponible derivados del ledger, intereses devengados, progreso, estado (ACTIVO, COMPLETADO, CANCELADO).
Simulacion	Cuota mensual, total aportado, intereses proyectados, monto final y proyección mes a mes. Pública y cacheable.
TablaTarifas	Tramos de tasa por plazo y bono por bloqueo.
Cuota	Calendario de débitos con estado (PROGRAMADA, PENDIENTE, EJECUTADA, CANCELADA) e intentos (máximo 5).
Aporte / Retiro	Movimiento de dinero asíncrono con la cuenta del Core, su estado, el motivo de fallo y el <code>correlationId</code> .
Movimiento	Asiento del ledger de solo inserción, con el saldo resultante.
CuentaDebito	Cuenta del cliente en el Core, enmascarada, con su saldo disponible.
CorridaCobro	Resultado de cada corte diario del Batch (para operación y auditoría).
Cliente / Token	Perfil del cliente y par de tokens JWT.
Problema	Detalle de error RFC 9457 con <code>codigo</code> y errores por campo.

La persistencia, en cambio, está normalizada en PostgreSQL: `planes`, `cuotas`, `operaciones_core`, el ledger `movimientos` (protegido por un trigger que rechaza `UPDATE` y `DELETE`), `outbox`, `eventos_procesados`, `idempotencia` y `corridas_cobro`. La vista `vw_planes` deriva el saldo, los intereses y el monto reservado por retiros pendientes, y el procedimiento `sp_procesar_debitos_ahorro_programado` solo selecciona las cuotas a cobrar.

4.4 Operaciones del contrato

La Tabla 4.3 lista las 20 operaciones del contrato con el scope que exige cada una y los códigos de estado que documenta. Se genera automáticamente desde el archivo OpenAPI, así que no puede quedar desactualizada respecto del contrato.

Tabla 4.3: *Operaciones del contrato OpenAPI (generada desde `contracts/openapi.yaml`)*

Método	Ruta	Operación	Scope requerido	Respuestas
POST	<code>/v1/auth/token</code>	Emitir o renovar un token	público	200, 400, 401, 429, 500
GET	<code>/v1/tarifas</code>	Consultar las tarifas vigentes	público	200, 429, 500
GET	<code>/v1/simulaciones</code>	Simular el rendimiento de un plan	público	200, 400, 429, 500

Método	Ruta	Operación	Scope requerido	Respuestas
GET	/v1/clientes/me	Consultar el perfil del cliente autenticado	perfil:leer	200, 401, 403, 429, 500
GET	/v1/cuentas-debito	Listar las cuentas del cliente	cuentas:leer	200, 401, 403, 429, 500, 503
GET	/v1/planes-ahorro	Listar planes con filtros	planes:leer	200, 400, 401, 403, 429, 500
POST	/v1/planes-ahorro	Crear un plan de ahorro	planes:escribir	201, 400, 401, 403, 409, 429, 500, 503
GET	/v1/planes-ahorro/{planId}	Consultar un plan	planes:leer	200, 401, 403, 404, 429, 500
PATCH	/v1/planes-ahorro/{planId}	Cambiar los datos descriptivos o el día de débito	planes:escribir	200, 400, 401, 403, 404, 409, 412, 415, 429, 500
POST	/v1/planes-ahorro/{planId}/bloqueo	Bloquear el plan para obtener mejor tasa	planes:escribir	200, 401, 403, 404, 409, 429, 500
POST	/v1/planes-ahorro/{planId}/desbloqueo	Desbloquear el plan antes de tiempo	planes:escribir	200, 401, 403, 404, 409, 429, 500
POST	/v1/planes-ahorro/{planId}/cancelacion	Cancelar el plan y devolver los fondos	planes:escribir	200, 400, 401, 403, 404, 409, 429, 500
POST	/v1/planes-ahorro/{planId}/aportes	Solicitar un aporte fuera del calendario	aportes:escribir	202, 400, 401, 403, 404, 409, 429, 500
GET	/v1/planes-ahorro/{planId}/aportes/{aporteId}	Consultar el estado de un aporte	planes:leer	200, 401, 403, 404, 429, 500
POST	/v1/planes-ahorro/{planId}/retiros	Retirar parte del saldo a una cuenta	retiros:escribir	202, 400, 401, 403, 404, 409, 429, 500
GET	/v1/planes-ahorro/{planId}/retiros/{retiroId}	Consultar el estado de un retiro	planes:leer	200, 401, 403, 404, 429, 500
GET	/v1/planes-ahorro/{planId}/cuotas	Consultar el calendario de cuotas	planes:leer	200, 400, 401, 403, 404, 429, 500
GET	/v1/planes-ahorro/{planId}/movimientos	Consultar los movimientos del plan	movimientos:leer	200, 400, 401, 403, 404, 429, 500
GET	/v1/corridas-cobro	Listar las corridas del corte diario de débitos	corridas:leer	200, 400, 401, 403, 429, 500
GET	/v1/corridas-cobro/{corridaId}	Consultar una corrida	corridas:leer	200, 401, 403, 404, 429, 500

4.5 Ajuste del contrato al frontend

Al revisar el frontend que construyó el equipo, ajustamos el contrato y el alcance: el login pasó a ser por email; la creación del plan usa una fecha objetivo y el sistema calcula la cuota; se agregaron el retiro parcial (RF-04.3) y el desbloqueo con pérdida de los intereses devengados (RF-01.7), con el evento **CreditoSolicitado** simétrico al débito; y se incorporaron el perfil del cliente, el saldo de las cuentas y un resumen de totales en el listado de planes. Estos cambios se reflejaron en los requerimientos, en la Fase 2 y en el diagrama C4.

4.6 Validación

El contrato se valida con Redocly sin errores (Anexo B). Las dos advertencias restantes corresponden a la URL de servidor local (`localhost`), que se mantiene a propósito para el entorno de desarrollo.

CAPÍTULO 5

Fase 4: desarrollo, seguridad y despliegue

5.1 Introducción

En las fases anteriores definimos el negocio (Fase 1), la arquitectura y los patrones (Fase 2) y el contrato OpenAPI (Fase 3). En esta fase construimos el backend que implementa ese contrato, lo protegimos con JWT, lo probamos con pruebas unitarias, de integración y de carga, y lo desplegamos en AWS con un pipeline de CI/CD.

El código está en la carpeta **backend/** del repositorio, el contrato en **contracts/openapi.yaml** los scripts de carga en **backend/pruebas-carga/** y todas las evidencias en **docs/evidencias/**.

5.2 Implementación del backend

5.2.1 Tecnología y organización por capas

El backend es un solo proyecto en **Node.js con TypeScript** (Express 5, Zod, PostgreSQL 16, RabbitMQ), organizado en cuatro capas cuyas dependencias apuntan hacia el dominio:

Tabla 5.1: *Capas del backend*

Capa	Carpeta	Contenido
Dominio	src/domain	Reglas puras: tarifas, simulación, estados del plan y de la cuota, estrategias de interés y de salida del bloqueo, eventos
Aplicación	src/application	Casos de uso (servicios) y puertos: repositorios, unidad de trabajo y CoreBancarioPort
Infraestructura	src/infrastructure	PostgreSQL, RabbitMQ con outbox, Adaptador Core con decoradores, JWT y bcrypt
Presentación	src/presentation	Express: DTOs con Zod, middlewares, controladores y mappers a la forma del contrato

Siguiendo el monolito modular de la Fase 2, el mismo código se ejecuta como cinco procesos, cada uno en su contenedor: Ahorro Core API, Servicio de Autenticación, Batch Processor, Adaptador Core y Core simulado (este último fuera del alcance del producto, sobre MongoDB Atlas).

5.2.2 Patrones de diseño implementados

Tabla 5.2: *Patrones implementados*

Patrón	Implementación
Adapter	<code>CoreLegacyAdapter</code> (HTTP hacia el Core) y <code>CoreSimuladoAdapter</code> (en memoria, para pruebas) implementan <code>CoreBancarioPort</code>
Decorator	Retry con backoff y jitter → Circuit Breaker (opossum) → Logging → Timeout, compuestos sobre el adaptador. En el camino síncrono no se usa Retry
Strategy	<code>CalculoInteresStrategy</code> (tasa base o con bono) y <code>PoliticaSalidaBloqueoStrategy</code> (pérdida de intereses al desbloquear o cancelar)
State	Ciclo de vida del plan (con el bloqueo como sub-estado) y cobro de la cuota (5 intentos a 24 h y prórroga de un mes)
Observer	Eventos de dominio publicados en RabbitMQ y consumidos por tipo de evento
Transactional Outbox	El evento se guarda en la misma transacción que el cambio y un relay lo publica después del commit
Consumidor idempotente y DLQ	Los eventos repetidos se descartan por su id; un mensaje que falla dos veces pasa a una cola <code>.dlq</code>
Repository y Unit of Work	Puertos en la capa de aplicación, implementados sobre transacciones de PostgreSQL

5.2.3 Manejo de errores y logs

Todos los errores se manejan en un middleware central y salen en formato `application/problem+json` (RFC 9457) con un `codigo` estable, como define el contrato. Cada petición lleva un `X-Request-Id` que se registra en los logs estructurados (pino), junto con el método, la ruta, el código de estado y el tiempo de respuesta. Las llamadas al Core también quedan registradas con su duración y resultado.

5.3 Seguridad

Tabla 5.3: *Mecanismos de seguridad*

Mecanismo	Implementación
Autenticación	Servicio de Autenticación propio: contraseñas con bcrypt, access token JWT RS256 de 15 minutos y refresh token con rotación (el anterior se invalida)
Validación del token	En el API Gateway (Kong, plugin JWT: firma y expiración) y de nuevo en la API (emisor, audiencia y claims), como defensa en profundidad
Autorización por rol y scope	Roles CLIENTE, OPERADOR y AUDITOR; cada endpoint exige un scope granular (<code>planes:leer</code> , <code>retiros:escribir</code> , <code>corridas:leer...</code>). Sin el scope, 403
Titularidad (ABAC)	Un cliente solo accede a sus planes; un plan ajeno responde 404 para no revelar que existe
Transporte	HTTPS en el Gateway solo con TLS 1.3 (perfil <code>modern</code>); una conexión con TLS 1.2 se rechaza
Abuso y fuerza bruta	Rate limiting en Kong: 30/min en el login, 600/min en los endpoints públicos y límite general en los autenticados
Integridad de operaciones	<code>Idempotency-Key</code> en las operaciones que mueven dinero; <code>ETag/If-Match</code> para modificar planes
Auditoría	Ledger de solo inserción: un trigger de PostgreSQL rechaza UPDATE y DELETE
Secretos	Claves JWT, contraseñas y la cadena de Atlas cifradas en AWS SSM Parameter Store; nunca en el repositorio ni en la imagen
Infraestructura	Sin SSH (puerto 22 cerrado): administración con SSM; el pipeline usa un usuario IAM con permisos mínimos

5.4 Pruebas unitarias y de integración

Escribimos 97 pruebas con Vitest:

- **84 unitarias** (sin dependencias externas): dominio (estados, estrategias, simulación), servicios sobre repositorios en memoria, decoradores de resiliencia (Retry, Timeout, Circuit Breaker), adaptadores del Core y la API HTTP completa con Supertest y tokens RS256 reales, incluida la publicación de la documentación.

- **13 de integración:** repositorios de PostgreSQL (transacciones, rollback, ledger inmutable, procedimiento del corte diario) y RabbitMQ (enrutamiento por evento y DLQ).

Tabla 5.4: Cobertura de código (umbral configurado: 80 %)

Métrica	Cobertura
Líneas	97,95 %
Sentencias	96,57 %
Funciones	97,47 %
Ramas	85,57 %

Se excluyen de la medición los puntos de entrada de los procesos, verificados con el despliegue, y el Core simulado, que está fuera del alcance. El reporte HTML de cobertura queda como artefacto en cada ejecución del pipeline.

5.5 Despliegue y CI/CD

5.5.1 Infraestructura en AWS

La cuenta del proyecto solo permite la región **us-east-2 (Ohio)** y tipos de instancia de la capa gratuita.

Tabla 5.5: Recursos en AWS

Recurso	Uso
EC2 m7i-flex.large (2 vCPU, 8 GB), Ubuntu 24.04	Ejecuta los 8 contenedores con Docker Compose
Elastic IP 3.151.57.252	URL pública fija de la API
Grupo de seguridad	Solo puertos 80 y 443 hacia Kong
ECR	Repositorio de imágenes de Docker
SSM Parameter Store	Secretos cifrados
MongoDB Atlas	Base del Core bancario simulado

La instancia se mantiene apagada fuera de las pruebas para no generar costos, y se enciende desde GitHub Actions cuando hace falta.

5.5.2 Pipeline de CI/CD

El workflow `.github/workflows/backend-ci-cd.yml` se ejecuta en cada push a main:

1. **Pruebas y contrato:** verificación de tipos, lint del contrato OpenAPI y las 97 pruebas con cobertura, usando PostgreSQL y RabbitMQ como servicios del runner.

2. **Imagen en ECR:** construye la imagen `linux/amd64` y la publica con el SHA del commit.
3. **Despliegue en EC2:** por SSM Run Command la instancia descarga la imagen, extrae de ella los archivos de despliegue, lee los secretos de SSM, levanta los contenedores y verifica la API. Termina con una prueba pública. Si la instancia está apagada, la imagen queda publicada y el despliegue se omite.

Un segundo workflow, `aws-instancia.yml`, enciende, apaga o consulta la instancia.

La documentación del contrato se publica junto con la API: Swagger UI en <http://3.151.57.252/docs/> y el archivo en `/docs/openapi.yaml`.

5.6 Pruebas de carga y estrés (k6)

5.6.1 Entorno y metodología

- **Generador de carga:** k6 2.3 desde un equipo en Ecuador, a través de Internet hacia la API en Ohio. Cada medición incluye la latencia de red de ida y vuelta (unos 100 ms, el mínimo observado), así que los tiempos son los que percibe un cliente real.
- **Sistema bajo prueba:** el despliegue de producción descrito en la sección 5, a través de Kong.
- **Mezcla de tráfico** de un cliente autenticado: listado de planes (30 %), detalle (20 %), movimientos con cursor (18 %), cuotas (12 %), cuentas en el Core por el camino síncrono hacia MongoDB Atlas (12 %) y aportes (8 %), con 0,5 a 1,5 s de pausa entre operaciones.
- **Recursos del servidor:** se registraron CPU y memoria de cada contenedor durante cada prueba.
- **Scripts y resultados:** `backend/pruebas-carga/` contiene los scripts y el analizador (`analizar.py`); las gráficas, los dashboards HTML de k6 y las métricas están en `docs/evidencias/fase-4-carga/`.

Antes de las pruebas, una medición inicial mostró que listar las cuentas tardaba 255 ms dentro del servidor, porque el Core simulado hacía dos consultas seguidas a Atlas. Las unimos en una sola consulta con `$lookup` y el tiempo bajó a unos 133 ms.

5.6.2 Escenario 1: carga sostenida

Subida de 0 a 200 usuarios virtuales en 2,5 min, meseta de 7 min y bajada en 1,5 min.

Tabla 5.6: *KPIs de la carga sostenida*

KPI	Resultado	Objetivo
Peticiones totales	90 815	—
Throughput en la meseta	167 RPS	—
Tiempo promedio	188 ms	—

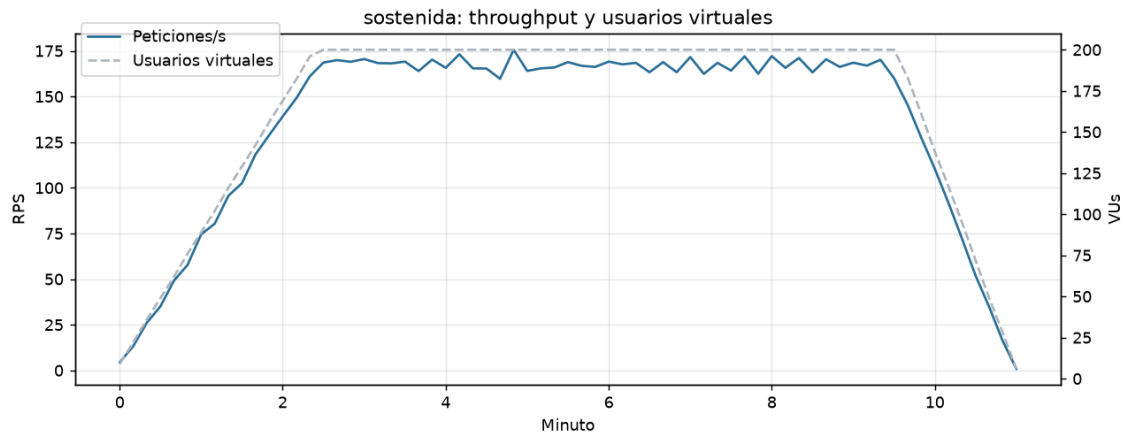


Figura 5.1: *Throughput y usuarios virtuales*

KPI	Resultado	Objetivo
Mediana	170 ms	—
p90	296 ms	—
p95	336 ms	< 500 ms ✓
p99	436 ms	—
Tasa de error	0,001 % (1 respuesta 502 de 90 815)	< 1 % ✓

Tabla 5.7: *Tiempos por endpoint en la carga sostenida*

End-point	Peticiones	Promedio (ms)	p90	p95	p99	Error %
GET /v1/planes-ahorro	27 258	178	272	306	399	0,00
GET /v1/planes-ahorro/{id}	18 172	163	250	278	338	0,00
GET .../movimientos	16 391	172	263	295	389	0,00
GET .../cuotas	11 118	171	264	300	396	0,00
GET /v1/cuentas-debito	10 757	298	394	431	480	0,00
POST .../aportes	7 108	184	289	333	462	0,01

Análisis. La API se mantuvo estable durante toda la meseta, con el p95 entre 300 y 420 ms, y cumplió los dos umbrales de la rúbrica. El endpoint más lento es el que consulta el Core por el camino síncrono (p95 de 431 ms), porque incluye la ida a MongoDB Atlas;

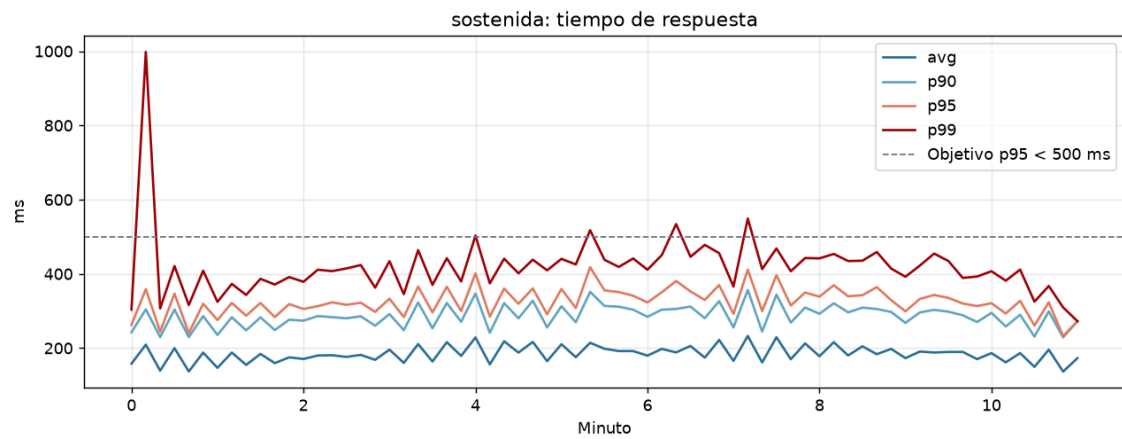


Figura 5.2: *Tiempo de respuesta*

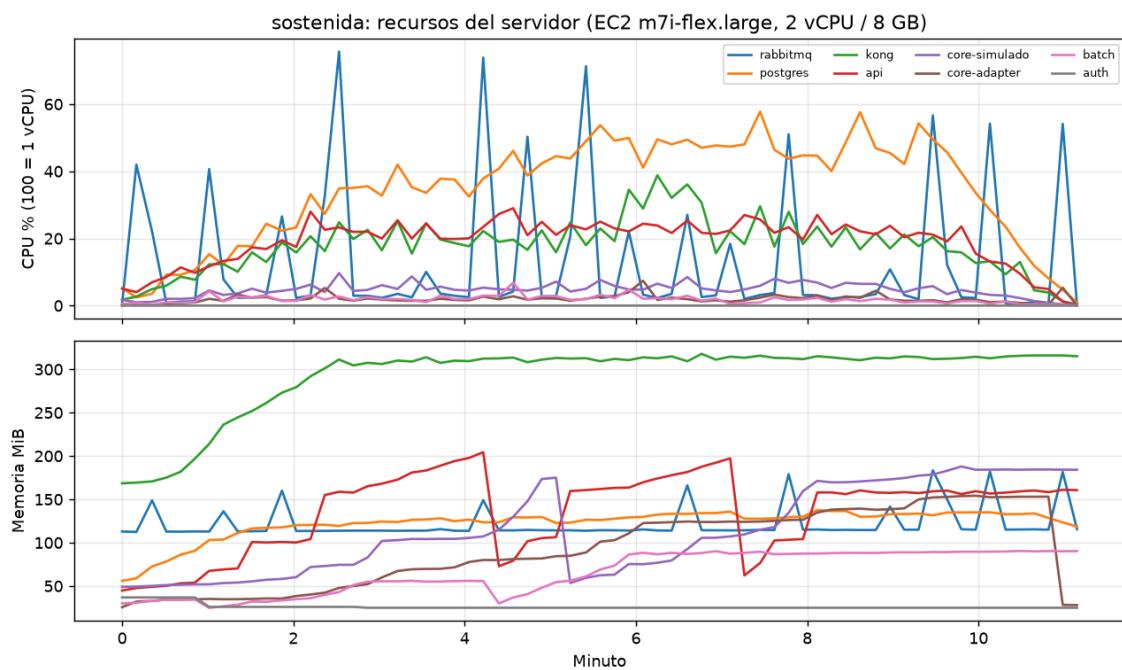


Figura 5.3: *Recursos del servidor*

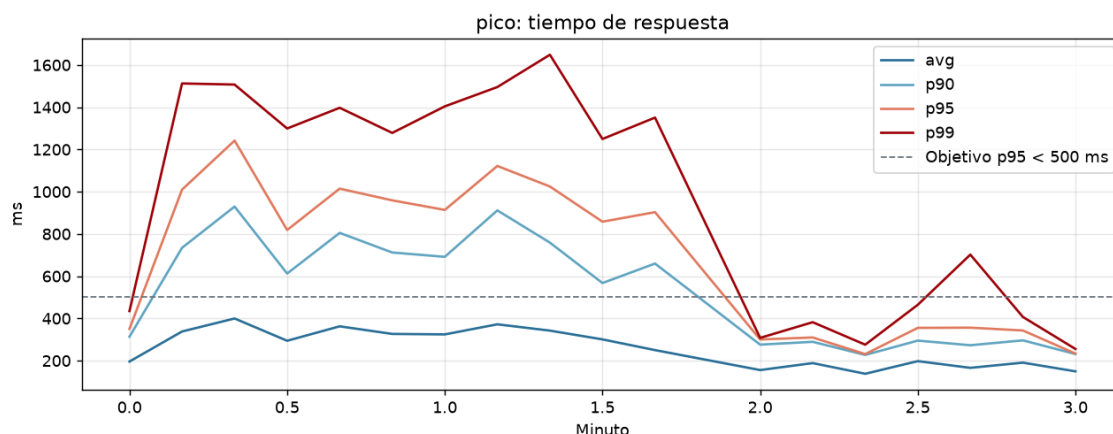


Figura 5.4: *Tiempo de respuesta en el pico*

aun así quedó bajo el objetivo. Los recursos quedaron holgados: PostgreSQL promedió 34 % de una vCPU y la API 19 %, con picos de memoria de 318 MiB en Kong y 204 MiB en la API. Hubo una sola respuesta 502 aislada en toda la prueba (0,001 %).

5.6.3 Escenario 2: pico extremo

De 0 a 500 usuarios virtuales en 10 s (450 clientes autenticados y 50 visitantes anónimos que simulan sin pausa), 1,5 min de pico, caída inmediata y 1 min de recuperación con 20 usuarios.

Tabla 5.8: *KPIs de la prueba de pico*

Grupo	Peticiones	RPS	Promedio	p95	p99	Errores
Clientes durante el pico	28 558	272	544 ms	1 249 ms	1 525 ms	0,00 %
Anónimos en /v1/simulaciones	33 421	321	145 ms	216 ms	283 ms	96,3 % (429, rate limiting)
Recuperación	1 034	17	172 ms	307 ms	453 ms	0,00 %

Análisis.

- **Sin errores bajo el pico:** con 450 clientes simultáneos (más del doble de la meseta de la carga sostenida y 22 veces la carga de recuperación), la API no devolvió errores. La latencia subió (p95 de 1,25 s) porque PostgreSQL llegó al 96,5 % de CPU, pero ninguna petición se perdió.
- **Rate limiting:** los visitantes anónimos superaron el límite de 600 peticiones por minuto por IP y Kong respondió 429 con **Retry-After** al 96 % de ellas, sin que esa carga llegara a la API. Las simulaciones que sí pasaron se sirvieron desde la caché del Gateway (145 ms en promedio).

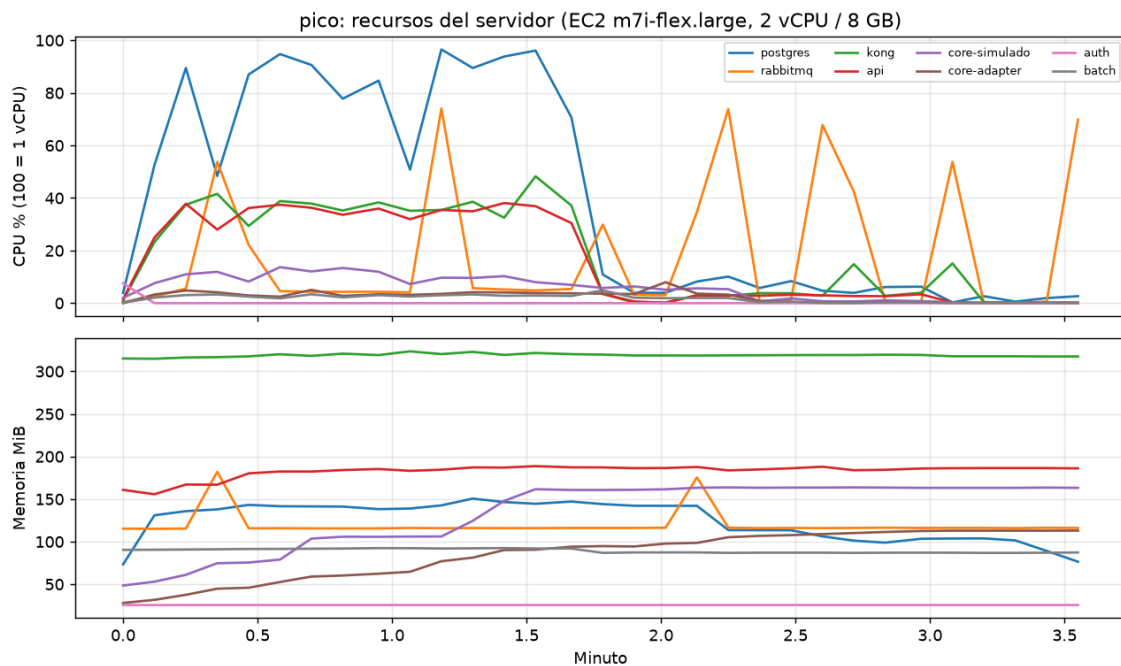


Figura 5.5: Recursos en el pico

- **Recuperación automática:** al terminar el pico, el sistema volvió solo a su comportamiento normal (p95 de 307 ms, 0 % de errores), sin reinicios ni caídas en cascada.

5.6.4 Escenario 3: punto de ruptura

La tasa de llegada sube de forma continua de 20 a 1 200 peticiones por segundo, sin pausas. La prueba se detiene sola cuando el p95 supera 1,5 s o los errores pasan del 5 %.

Tabla 5.9: Evolución hacia el punto de ruptura

Momento	RPS	Peticiones en curso	p95	Errores
Funcionamiento normal	144	40	307 ms	0 %
Último punto dentro del objetivo	204	68	428 ms	0 %
Degradación (p95 > 500 ms sostenido)	221	138	588 ms	0 %
Saturación (máximo throughput)	302	210	794 ms	0 %

Momento	RPS	Peticiones en curso	p95	Errores
Corte de la prueba	269	456	2 153 ms	0 %

Análisis. Con el despliegue actual en una sola instancia de 2 vCPU:

- **Punto de ruptura:** ~220 RPS (unas 140 peticiones simultáneas). Desde ahí el p95 deja de cumplir los 500 ms.
- **Capacidad máxima:** ~300 RPS. Más allá, el throughput ya no crece: la cola de peticiones aumenta y la latencia sube de forma exponencial, hasta un p95 de 2,1 s.
- **Cuello de botella:** PostgreSQL llegó al 98,5 % de CPU en el mismo momento, mientras la API (39 % máximo) y Kong (43 %) seguían con margen.
- **Degradación sin fallos:** aun saturado, el sistema casi no devolvió errores (0,018 %). Se degrada por latencia, no por fallos, lo que es preferible porque los reintentos del cliente no empeoran la situación.

5.6.5 Conclusiones y mejoras propuestas

1. **Cumplimiento:** la API cumple los objetivos de la rúbrica bajo la carga sostenida esperada (p95 de 336 ms con 200 usuarios, error de 0,001 %) y tiene un margen de capacidad de ~30 % antes del punto de ruptura.
2. **Resiliencia demostrada:** rate limiting, caché del Gateway y recuperación sin intervención después de un pico de 500 usuarios simultáneos.
3. **Mejora para escalar:** el factor limitante es el cálculo de saldos en PostgreSQL. La vista `vw_planes` suma el ledger completo de cada plan en cada lectura, y ese costo crece con los movimientos. Proponemos mantener una tabla de saldos actualizada en la misma transacción que cada asiento, sin dejar de derivar del ledger, y escalar PostgreSQL a una instancia administrada (RDS). Con eso el siguiente límite pasaría a ser la API, que sí se puede replicar horizontalmente detrás de Kong (RNF-01.3).

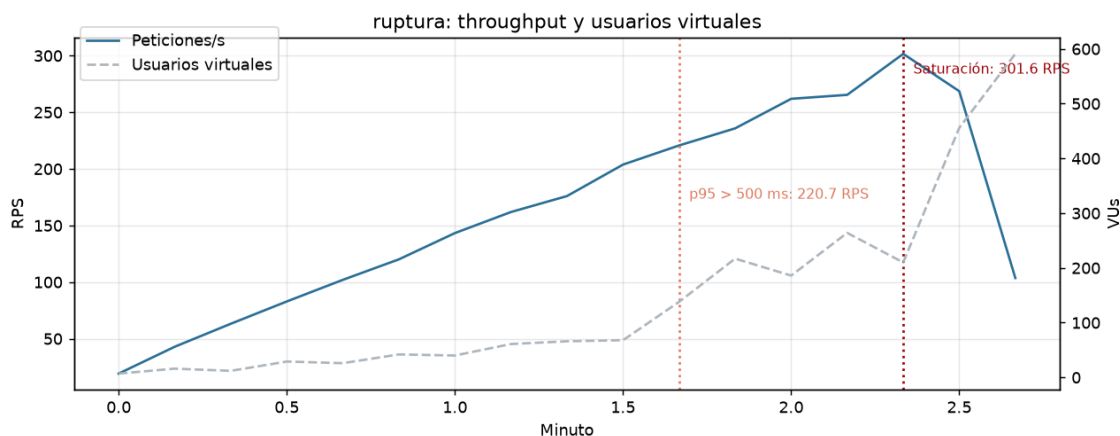


Figura 5.6: Punto de ruptura

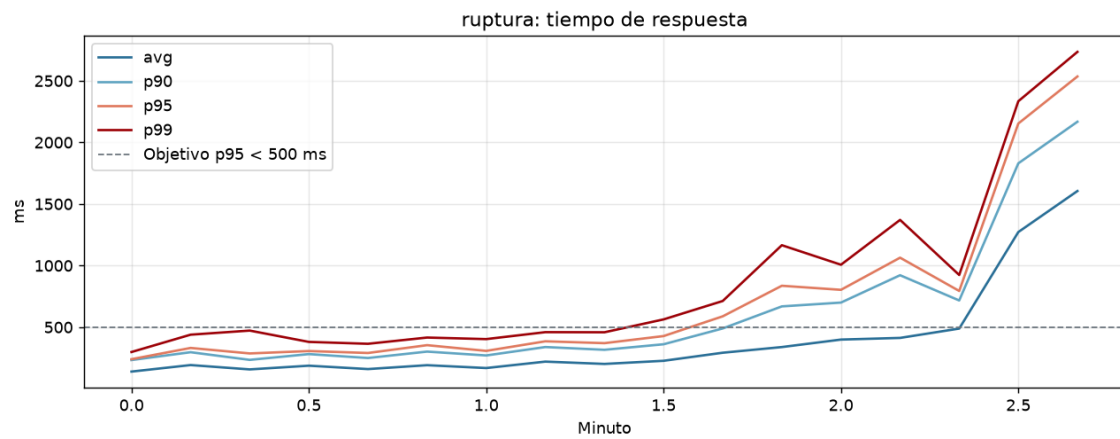


Figura 5.7: Latencia hacia la ruptura

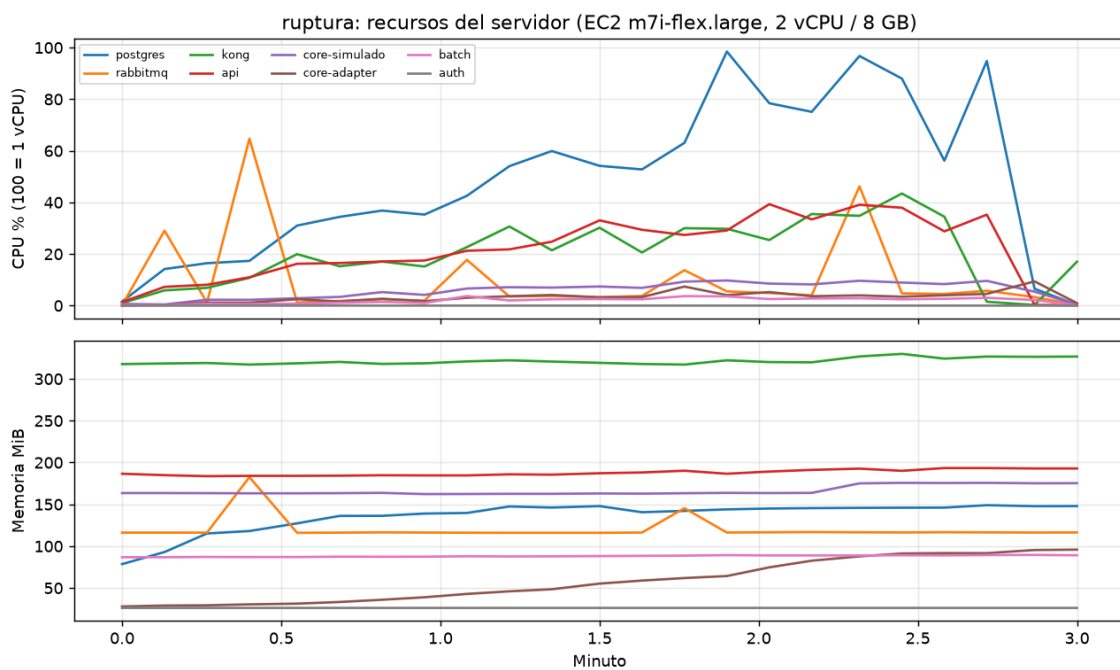


Figura 5.8: Recursos hacia la ruptura

5.7 Evidencias

Tabla 5.10: *Evidencias*

Carpeta	Contenido
<code>docs/evidencias/fase-3-contrato/</code>	Validación del contrato con Redocly y capturas de Swagger UI publicado
<code>docs/evidencias/fase-4-pruebas/</code>	Salida de las 97 pruebas, resumen de cobertura y reporte HTML
<code>docs/evidencias/fase-4-seguridad/</code>	18 verificaciones contra la API desplegada: 401, 403, 404, 400, idempotencia, 409, 412, 202, rotación del refresh token, TLS 1.3 y 429
<code>docs/evidencias/fase-4-despliegue/</code>	Recursos en AWS, contenedores en la instancia y ejecución del pipeline
<code>docs/evidencias/fase-4-carga/</code>	Resultados de k6, gráficas, dashboards HTML y métricas del servidor

CAPÍTULO 6

Integración con el frontend y pruebas de extremo a extremo

El frontend (Next.js, React y Zustand) ya sigue la forma del contrato: montos en centavos, los mismos estados del plan y los mismos códigos de error. Hasta ahora funciona con datos de prueba en el navegador. Este capítulo documentará su conexión con la API desplegada.

Pendiente

Se completará al terminar la integración:

- Reemplazo del almacenamiento local por llamadas HTTP a la API (login con JWT y renovación con refresh token, planes, aportes, retiros, bloqueo, desbloqueo y cancelación).
- Manejo de las operaciones asíncronas en la interfaz (estado PENDIENTE hasta la confirmación del Core).
- Manejo de los errores `application/problem+json` (mensajes al usuario según el código).
- Pruebas de extremo a extremo en el navegador contra el entorno de AWS, con capturas como evidencia.

CAPÍTULO 7

Conclusiones y trabajo futuro

7.1 Conclusiones

1. **El enfoque API-First ordenó el trabajo del equipo.** Con el contrato OpenAPI como fuente de verdad, el frontend y el backend avanzaron en paralelo, y los cambios de alcance se discutieron sobre el contrato antes de escribir código.
2. **Cada patrón responde a un requisito concreto.** El estilo híbrido REST y orientado a eventos resolvió el pico del corte diario y el desacoplamiento del Core; Adapter y Decorator aislaron el Core y su resiliencia; Strategy y State concentraron las reglas del negocio en código fácil de probar; el Transactional Outbox evitó perder eventos sin renunciar a la consistencia ACID del saldo.
3. **La calidad se pudo medir.** Las 97 pruebas cubren el 97,9 % de las líneas, y las pruebas de carga confirmaron los objetivos de la rúbrica: p95 de 336 ms y 0,001 % de errores con 200 usuarios simultáneos, recuperación automática después de un pico de 500 usuarios y un punto de ruptura identificado en unas 220 peticiones por segundo.
4. **Medir sacó a la luz problemas reales.** Las pruebas en el entorno desplegado revelaron una consulta al Core con dos viajes a la base, una caché de DNS del Gateway que provocaba errores 502 tras un redesplicue y una configuración de TLS que aceptaba TLS 1.2. Los tres se corrigieron y se verificaron.
5. **La automatización reduce el riesgo de cada cambio.** El pipeline ejecuta pruebas, valida el contrato, publica la imagen y despliega sin acceso SSH, con los secretos fuera del repositorio.

7.2 Trabajo futuro

- Mantener una tabla de saldos actualizada en la misma transacción que el ledger, para que las lecturas no recorran todos los movimientos, y migrar PostgreSQL a un servicio administrado.
- Escalar horizontalmente la API detrás del Gateway y separar la base de lecturas cuando el volumen lo justifique (CQRS).
- Incorporar un certificado TLS emitido por una autoridad reconocida y un dominio propio.
- Abrir el canal de socios (ahorro embebido) con una versión v2 del contrato y credenciales propias por socio.

Referencias

- Brown, S. (2026). *The C4 model for visualising software architecture*. <https://c4model.com>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Grafana Labs. (2026). *Grafana k6 documentation*. <https://grafana.com/docs/k6/>
- International Organization for Standardization. (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC N.º 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem Details for HTTP APIs* (RFC N.º 9457). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9457>
- Nygaard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2.^a ed.). Pragmatic Bookshelf.
- OpenAPI Initiative. (2021). *OpenAPI Specification, versión 3.1.0*. <https://spec.openapis.org/oas/v3.1.0>
- Prieto, P. M. (2026a). Arquitectura de seguridad en APIs.
- Prieto, P. M. (2026b). Unidad 1: Introducción a sistemas basados en APIs.
- Prieto, P. M. (2026c). Unidad 2: Arquitectura y patrones para sistemas dirigidos por APIs.
- Prieto, P. M. (2026d). Unidad 3: Diseño de APIs.
- Prieto, P. M. (2026e). Unidad 4: Desarrollo de sistemas basados en APIs.
- Richardson, C. (2018). *Microservices Patterns*. Manning.

APÉNDICE A

Detalle de los patrones de diseño

A.1 Adapter

Dónde: contenedor **Adaptador Core (Fachada)**. Se define la interfaz `CoreBancarioPort` con dos implementaciones, seleccionadas por configuración (`CORE_MODO`):

- **CoreLegacyAdapter:** traduce las llamadas del dominio a la interfaz SOAP/REST del Core Bancario Legacy. En el proyecto, ese Core es la simulación en MongoDB de la carpeta `bdd/` (fuera de alcance).
- **CoreSimuladoAdapter:** responde en memoria, sin depender de ningún Core (perfil de pruebas unitarias).

Atiende los dos flujos del diagrama de contenedores: la validación síncrona de titularidad y saldo que pide la Ahorro Core API (REST) y el consumo de `DebitoSolicitado` y `CreditoSolicitado` (AMQP).

Por qué / impacto:

- Aísla el dominio del legacy: si el Core cambia, solo se modifica el adaptador (RNF-03.1).
- La implementación en memoria permite correr las pruebas unitarias sin depender del Core.

A.2 Decorator

Dónde: dentro del **Adaptador Core**, como envoltorios sobre `CoreBancarioPort`: `Retry( CircuitBreaker( Logging( Timeout( CoreLegacyAdapter ) ) ) )`. Cada decorador implementa `CoreBancarioPort`; el Circuit Breaker usa la librería `opossum`.

El Retry va por fuera para que cada intento cuente en la ventana del Circuit Breaker; con el circuito abierto, el Retry no reintenta y el fallo se devuelve de inmediato.

Por qué / impacto:

- Circuit Breaker y Retry con backoff exponencial y jitter (1 s, 2 s, 4 s + aleatorio) se agregan por composición, sin modificar el adaptador (RNF-03.2). Corresponde al elemento “Circuit Breaker + Retry” de la vista ArchiMate.
- Cada capa de resiliencia se puede probar de forma aislada.
- La capa de logging registra errores y tiempos de respuesta, que la rúbrica pide.
- La política depende del camino (Fase 2, §4.2):

- **Síncrono** (validación de titularidad y saldo, el cliente espera): timeout de 2 s y Circuit Breaker, **sin** Retry con espera. Con el backoff completo, una validación fallida tardaría más de 7 s y rompería el p95 < 500 ms.
 - **Asíncrono** (consumo de `DebitoSolicitado` y `CreditoSolicitado`): Circuit Breaker y Retry con backoff y jitter.
- Solo cubre el reintento **técnico**. El reintento **de negocio** (5 intentos cada 24 h) lo gestiona el Batch Processor (ver patrón State).

A.3 Strategy

Dónde: Ahorro Core API, en el dominio del plan:

- `CalculoInteresStrategy`: tasa base o tasa con bono por bloqueo (RF-01.6).
- `PoliticaSalidaBloqueoStrategy`: al cancelar o desbloquear, sin penalidad o con pérdida de los intereses devengados si el plan está bloqueado (RF-01.7, RF-04.1, RF-04.2).

Por qué / impacto:

- La simulación pública (RF-02.1) y el plan real usan el mismo cálculo, así que el valor simulado coincide con el real.
- Elimina las reglas duplicadas que hoy están en el frontend (`frontend/lib/store.ts`) y en los scripts de base de datos. Las tasas pasan a vivir solo en el backend.
- Son clases puras, fáciles de cubrir con pruebas unitarias (meta > 80 %).

A.4 State

Dónde:

- **Ahorro Core API**, para el ciclo de vida del plan: `ACTIVO` → `COMPLETADO` y `ACTIVO` → `CANCELADO` (RF-01.4), con el sub-estado de bloqueo dentro de `ACTIVO` (bloqueado ↔ no bloqueado; un plan bloqueado no admite retiros).
- **Batch Processor**, para el ciclo de vida del débito: `PENDIENTE` → `EJECUTADO` o `PENDIENTE` → `FALLIDO`. Si el Core rechaza un débito automático, se reprograma a las 24 h, hasta un máximo de 5 intentos (RF-03.1). Al quinto fallo la cuota se cancela y **el plazo del plan se extiende un mes** (regla de cobranza de la Fase 1). Un aporte bajo solicitud no se reprograma: queda `FALLIDO` y el cliente decide si lo vuelve a pedir.

Por qué / impacto:

- Impide transiciones inválidas, como aportar a un plan cancelado, cancelar uno completado o retirar de uno bloqueado.
- Cada transición inválida se traduce directamente en un `409 Conflict` del contrato OpenAPI.
- La política de 5 intentos queda en un solo lugar y no repartida en condicionales.

- Expone al cliente el estado del débito (PENDIENTE, EJECUTADO, FALLIDO), que es la mitigación de consistencia eventual definida en la Fase 2 (§3.3).

A.5 Observer (eventos de dominio)

Dónde: según el diagrama de contenedores, los eventos viajan por RabbitMQ así:

Publica	Evento	Consume
Ahorro Core API	PlanCreado	Sistema de Notificaciones
Ahorro Core API	PlanCancelado	Sistema de Notificaciones
Ahorro Core API (aporte bajo solicitud) y Batch Processor (débito automático)	DebitoSolicitado	Adaptador Core
Ahorro Core API (retiro parcial y devolución al cancelar)	CreditoSolicitado	Adaptador Core
Adaptador Core	DebitoEjecutado, DebitoFallido, CreditoEjecutado, CreditoFallido	Batch Processor (registra en el ledger el resultado de todo débito y crédito y aplica los reintentos), Sistema de Notificaciones

Dentro de cada servicio, los eventos se crean como eventos de dominio y se guardan en la outbox; un relay los publica en RabbitMQ y los consumidores se suscriben por tipo de evento. Si un consumidor falla dos veces, el mensaje pasa a una Dead Letter Queue.

Por qué / impacto:

- Desacopla la API del Core legacy y de las notificaciones (RNF-03.1, RF-05.2).
- Absorbe el pico del corte diario de débitos (RNF-01.2).
- Los dos tipos de débito comparten el mismo evento, así que la resiliencia y la auditoría se implementan una sola vez.
- **Condición obligatoria (Transactional Outbox):** en la Ahorro Core API y el Batch Processor, que son los que escriben en PostgreSQL, el evento se guarda en una tabla **outbox** dentro de la misma transacción que la operación que lo origina (crear o cancelar el plan, registrar el débito como PENDIENTE). Se publica después del commit. Así nunca queda un cambio guardado sin su evento, ni un evento publicado de un cambio que no se guardó (RNF-02.3).
- Como el broker puede entregar un mensaje más de una vez, los consumidores deben ser idempotentes. El **correlation-id** del evento sirve para detectar duplicados.

A.6 Requisito para aplicar State y Strategy en el Batch

El diagrama C4 indica que el Batch Processor invoca el stored procedure `sp_procesar_debitos_ahorro_programado()`. Para que los patrones anteriores apliquen, el SP solo debe **seleccionar los débitos del día**. Las transiciones de estado, los reintentos y los cálculos deben quedar en el código del Batch. Si esa lógica se deja en el SP, los patrones State y Strategy no aplican en el Batch y esa parte queda fuera de la cobertura de pruebas unitarias.

APÉNDICE B

Evidencias

Todas las evidencias están en `docs/evidencias/` del repositorio. Se pueden regenerar con los scripts indicados en cada sección.

B.1 Validación del contrato

```
No configurations were provided -- using built in recommended configuration by default.
```

```
validating contracts/openapi.yaml...
```

```
[1] contracts/openapi.yaml:3:1 at #/info
```

```
Info object should contain `license` field.
```

```
1 | openapi: 3.1.0
```

```
2 |
```

```
3 | info:
```

```
  | ~~~~
```

```
4 |   title: Billetera de Ahorro API
```

```
5 |   version: 1.0.0
```

```
Warning was generated by the info-license rule.
```

```
Reference: https://redocly.com/docs/cli/rules/oas/info-license
```

```
[2] contracts/openapi.yaml:62:10 at #/servers/2/url
```

```
Server `url` should not point to example.com or localhost.
```

```
60 | - url: http://3.151.57.252
```

```
61 |   description: Producción en AWS (Kong, HTTP)
```

```
62 | - url: http://localhost:8000
```

```
  | ~~~~~
```

```
63 |   description: Local (Kong en Docker Compose)
```

```
64 |
```

```
Warning was generated by the no-server-example.com rule.
```

```
Reference: https://redocly.com/docs/cli/rules/oas/no-server-example-com
```

```
contracts/openapi.yaml: validated in 114ms
```

```
Woohoo! Your API description is valid.
```

```
You have 2 warnings.
```

B.2 Pruebas unitarias y de integración

```
Resultado de las pruebas (vitest run --coverage)
```

```
Test Files  6 passed (6)
```

```
Tests      97 passed (97)
```

```
Start at   11:16:42
```

```
Duration   3.39s (tests 69%, import 15%, transform 14%, worker 3%)
```


Pruebas ejecutadas:

```

|unit| test/unit/dominio.test.ts > fechas > suma meses conservando el día y ajusta al último día del mes
|unit| test/unit/dominio.test.ts > fechas > cuenta meses completos
|unit| test/unit/dominio.test.ts > fechas > calcula la próxima fecha con un día dado, siempre posterior
|unit| test/unit/dominio.test.ts > tarifas > elige el tramo por plazo y usa el último si no hay tope
|unit| test/unit/dominio.test.ts > tarifas > calcula la TEA con capitalización mensual
|unit| test/unit/dominio.test.ts > simulación (mismo ejemplo que el contrato) > meta de USD 1.200 a 12
    meses bloqueado: cuota 97,46
|unit| test/unit/dominio.test.ts > simulación (mismo ejemplo que el contrato) > rechaza montos y fechas
    fuera de rango
|unit| test/unit/dominio.test.ts > simulación (mismo ejemplo que el contrato) > con tasa 0 reparte la meta
    en partes iguales
|unit| test/unit/dominio.test.ts > Strategy: cálculo de interés > sin bloqueo usa la tasa base; con
    bloqueo suma el bono del tramo
|unit| test/unit/dominio.test.ts > Strategy: cálculo de interés > respeta la tasa base congelada del plan
|unit| test/unit/dominio.test.ts > Strategy: cálculo de interés > el interés mensual se redondea al
    centavo
|unit| test/unit/dominio.test.ts > Strategy: salida del bloqueo > solo un plan bloqueado pierde los
    intereses devengados
|unit| test/unit/dominio.test.ts > State: ciclo de vida del plan > ACTIVO permite operar y valida el sub-
    estado de bloqueo
|unit| test/unit/dominio.test.ts > State: ciclo de vida del plan > un plan bloqueado no admite retiros y
    el monto no puede superar el disponible
|unit| test/unit/dominio.test.ts > State: ciclo de vida del plan > se completa cuando el saldo alcanza la
    meta
|unit| test/unit/dominio.test.ts > State: ciclo de vida del plan > COMPLETADO es final: todo lanza
    ESTADO_INVALIDO
|unit| test/unit/dominio.test.ts > State: ciclo de vida del plan > CANCELADO es final: todo lanza
    ESTADO_INVALIDO
|unit| test/unit/dominio.test.ts > State: ciclo de vida del plan > progreso y saldo disponible
|unit| test/unit/dominio.test.ts > State: cobro de una cuota (5 intentos a 24 h) > PROGRAMADA → PENDIENTE
    → EJECUTADA
|unit| test/unit/dominio.test.ts > State: cobro de una cuota (5 intentos a 24 h) > un rechazo reprograma a
    las 24 h y el quinto cancela la cuota
|unit| test/unit/dominio.test.ts > State: cobro de una cuota (5 intentos a 24 h) > un fallo técnico
    reprograma sin consumir intentos
|unit| test/unit/dominio.test.ts > State: cobro de una cuota (5 intentos a 24 h) > transiciones inválidas
|unit| test/unit/dominio.test.ts > operaciones > clasifica débitos y arma la etiqueta de la cuenta
|unit| test/unit/servicios.test.ts > PlanesService > crea el plan con cuota calculada, calendario y evento
    PlanCreado en la outbox
|unit| test/unit/servicios.test.ts > PlanesService > rechaza una cuenta que no es del cliente y traduce la
    caída del Core a 503
|unit| test/unit/servicios.test.ts > PlanesService > lista con resumen; un cliente no puede filtrar por
    clienteId y solo ve lo suyo
|unit| test/unit/servicios.test.ts > PlanesService > ABAC: un plan ajeno responde NO_ENCONTRADO
|unit| test/unit/servicios.test.ts > PlanesService > modifica datos y reprograma las cuotas al cambiar el
    día; valida la versión
|unit| test/unit/servicios.test.ts > PlanesService > bloquea con bono; desbloquear pierde los intereses
    devengados (PENALIDAD en el ledger)
|unit| test/unit/servicios.test.ts > PlanesService > cancela: cancela cuotas, solicita la devolución y
    publica PlanCancelado
|unit| test/unit/servicios.test.ts > PlanesService > cancelar un plan sin saldo no solicita devolución
|unit| test/unit/servicios.test.ts > MovimientosDineroService + resultados del Core > aporte: queda
    PENDIENTE, el Core debita y el resultado se asienta en el ledger
|unit| test/unit/servicios.test.ts > MovimientosDineroService + resultados del Core > aporte rechazado por
    fondos insuficientes queda FALLIDO y no toca el saldo
|unit| test/unit/servicios.test.ts > MovimientosDineroService + resultados del Core > retiro: reserva el
    monto, valida bloqueo y saldo, y se asienta al confirmarse
|unit| test/unit/servicios.test.ts > MovimientosDineroService + resultados del Core > al alcanzar la meta
    el plan se completa y se liquidan los fondos
|unit| test/unit/servicios.test.ts > MovimientosDineroService + resultados del Core > un débito confirmado
    después de cancelar se devuelve a la cuenta
|unit| test/unit/servicios.test.ts > MovimientosDineroService + resultados del Core > descarta eventos
    duplicados (consumidor idempotente) y resultados de operaciones inexistentes
|unit| test/unit/servicios.test.ts > Batch: corte diario y reintentos de negocio > cobra la cuota, devenga
    el interés del mes y cuenta la corrida
|unit| test/unit/servicios.test.ts > Batch: corte diario y reintentos de negocio > cinco rechazos cancelan
    la cuota y extienden el plazo un mes (prórroga)
|unit| test/unit/servicios.test.ts > Batch: corte diario y reintentos de negocio > un fallo técnico del
    Core no consume intentos
|unit| test/unit/servicios.test.ts > Batch: corte diario y reintentos de negocio > no cobra cuotas de
    planes cancelados y registra corridas fallidas

```

```

|unit| test/unit/servicios.test.ts > ConsultasService > pagina movimientos por cursor sin repetir ni
    saltar asientos
|unit| test/unit/servicios.test.ts > ClientesService > devuelve perfil y cuentas; un cliente sin Core no
    tiene cuentas
|unit| test/unit/servicios.test.ts > EjecutorCoreService > traduce crédito exitoso y fallido
|unit| test/unit/core.test.ts > RetryCoreDecorator (backoff exponencial con jitter) > espera base-2^n +
    jitter
|unit| test/unit/core.test.ts > RetryCoreDecorator (backoff exponencial con jitter) > reintenta fallos té
    cnicos hasta tener éxito
|unit| test/unit/core.test.ts > RetryCoreDecorator (backoff exponencial con jitter) > se rinde al agotar
    los intentos
|unit| test/unit/core.test.ts > RetryCoreDecorator (backoff exponencial con jitter) > no reintenta
    rechazos de negocio ni el circuito abierto
|unit| test/unit/core.test.ts > RetryCoreDecorator (backoff exponencial con jitter) > usa la espera real
    por defecto
|unit| test/unit/core.test.ts > TimeoutCoreDecorator > corta una llamada lenta con CoreNoDisponibleError
|unit| test/unit/core.test.ts > TimeoutCoreDecorator > deja pasar respuestas a tiempo
|unit| test/unit/core.test.ts > CircuitBreakerCoreDecorator > se abre con fallos técnicos y responde de
    inmediato mientras está abierto
|unit| test/unit/core.test.ts > CircuitBreakerCoreDecorator > los rechazos de negocio no abren el circuito

|unit| test/unit/core.test.ts > CircuitBreakerCoreDecorator > pasa a semiabierto después del tiempo de
    reset
|unit| test/unit/core.test.ts > LoggingCoreDecorator > registra éxito, rechazo y error con su duración
|unit| test/unit/core.test.ts > LoggingCoreDecorator > delega todas las operaciones del puerto
|unit| test/unit/core.test.ts > CoreSimuladoAdapter > debita, acredita, rechaza y es idempotente por
    operación
|unit| test/unit/core.test.ts > CoreLegacyAdapter (HTTP) > traduce cuentas del Core (decimales) a centavos
    y enmascara el número
|unit| test/unit/core.test.ts > CoreLegacyAdapter (HTTP) > 404 → sin cuentas o cuenta inexistente
|unit| test/unit/core.test.ts > CoreLegacyAdapter (HTTP) > obtiene una cuenta
|unit| test/unit/core.test.ts > CoreLegacyAdapter (HTTP) > envía el monto en decimales y traduce rechazos
    422
|unit| test/unit/core.test.ts > CoreLegacyAdapter (HTTP) > 5xx y errores de red son fallos técnicos
|unit| test/unit/core.test.ts > fabrica del puerto del Core > el camino síncrono no reintenta y el así
    ncrono sí
|unit| test/unit/core.test.ts > fabrica del puerto del Core > valida la configuración
|unit| test/unit/http.test.ts > endpoints públicos > GET /v1/tarifas y /v1/simulaciones son anónimos y
    cacheables
|unit| test/unit/http.test.ts > endpoints públicos > valida la query y responde problem+json
|unit| test/unit/http.test.ts > endpoints públicos > respeta el X-Request-Id recibido, expone /health y
    responde 404 a rutas inexistentes
|unit| test/unit/http.test.ts > documentación > publica el contrato OpenAPI y Swagger UI sin autenticación

|unit| test/unit/http.test.ts > seguridad > sin token, con token inválido o expirado responde 401 con WWW-
    Authenticate
|unit| test/unit/http.test.ts > seguridad > exige el scope de la operación (403)
|unit| test/unit/http.test.ts > seguridad > un token sin clave privada no se puede firmar
|unit| test/unit/http.test.ts > planes > POST crea (201 + Location + ETag) y GET devuelve la forma del
    contrato
|unit| test/unit/http.test.ts > planes > Idempotency-Key: repite la respuesta original y rechaza otro
    cuerpo con 409
|unit| test/unit/http.test.ts > planes > valida el cuerpo, el JSON y el planId
|unit| test/unit/http.test.ts > planes > lista con filtros, orden y resumen
|unit| test/unit/http.test.ts > planes > PATCH exige merge-patch+json y controla la versión con If-Match
|unit| test/unit/http.test.ts > planes > bloqueo, desbloqueo, cancelación y conflictos de estado (409)
|unit| test/unit/http.test.ts > aportes, retiros, cuotas, movimientos y corridas > aporte 202 + Location;
    retiro 202; consultas
|unit| test/unit/http.test.ts > aportes, retiros, cuotas, movimientos y corridas > perfil y cuentas del
    cliente
|unit| test/unit/http.test.ts > aportes, retiros, cuotas, movimientos y corridas > un error inesperado
    responde 500 sin filtrar detalles
|unit| test/unit/http.test.ts > Servicio de Autenticación > emite tokens con la contraseña y rota el
    refresh token
|unit| test/unit/http.test.ts > Servicio de Autenticación > rechaza credenciales inválidas, usuarios
    inexistentes, refresh vencidos y cuerpos inválidos
|unit| test/unit/http.test.ts > Servicio de Autenticación > scopes por rol y combinación de roles
|unit| test/unit/http.test.ts > utilidades de presentación > problem details y cursores
|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
    outbox > crea, aporta, retira, bloquea, desbloquea y cancela con saldos derivados del ledger
|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
    outbox > el ledger es de solo inserción (el trigger rechaza UPDATE y DELETE)

```

```

|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
  outbox > si la transacción falla no queda ni el cambio ni el evento (rollback)
|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
  outbox > listar: filtros, orden por progreso y resumen sobre todos los planes
|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
  outbox > modificar reprograma cuotas y respeta la versión
|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
  outbox > corte diario: el SP selecciona cuotas y reintentos; rechazos, prórroga y corridas
|integration| test/integration/postgres.test.ts > PostgreSQL: flujo completo con transacciones, ledger y
  outbox > completar el plan liquida los fondos; operaciones y eventos repetidos se ignoran
|integration| test/integration/postgres.test.ts > PostgreSQL: autenticación, idempotencia y outbox >
  refresh tokens y usuarios
|integration| test/integration/postgres.test.ts > PostgreSQL: autenticación, idempotencia y outbox >
  respuestas idempotentes se guardan y se reemplazan
|integration| test/integration/postgres.test.ts > PostgreSQL: autenticación, idempotencia y outbox > el
  relay deja el evento pendiente si el broker falla
|integration| test/integration/rabbitmq.test.ts > RabbitMQ: publicación/suscripción por tipo de evento >
  enruta cada evento a las colas suscritas y entrega al consumidor
|integration| test/integration/rabbitmq.test.ts > RabbitMQ: publicación/suscripción por tipo de evento >
  un mensaje que falla dos veces termina en la DLQ (no se pierde)
|integration| test/integration/rabbitmq.test.ts > RabbitMQ: publicación/suscripción por tipo de evento >
  falla al conectar si el broker no existe

```

Cobertura (archivos con menos de 100 % en alguna métrica):

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
-----	-----	-----	-----	-----	-----
All files	96.57	85.57	97.47	97.95	
application/ports	60	100	50	75	
servicios.ts	0	100	0	0	29-30
...ation/services	97.17	88.08	100	98.62	
auth.service.ts	94.59	86.66	100	96.87	61
...os.service.ts	89.47	77.77	100	94.28	101-102
...es.service.ts	100	92.3	100	100	147-157,192
...re.service.ts	96.05	79.41	100	98.43	119
config	75	63.63	50	80	
env.ts	75	63.63	50	80	62-63
domain	96.87	90	100	96.72	
eventos.ts	0	100	100	0	46-47
fechas.ts	100	70	100	100	5
...ture/messaging	88.31	70	81.25	94.11	
outbox-relay.ts	87.09	66.66	83.33	92.85	59-60
rabbitmq.ts	89.13	71.42	80	95	46-47
...cture/postgres	93.54	100	81.81	100	
...it-of-work.ts	92.3	100	75	100	
pool.ts	94.44	100	85.71	100	
...s/repositories	97.82	69.07	100	98.38	
...repository.ts	98.3	60.52	100	100	...,85-99,158-276
...repository.ts	94.28	70	100	92.85	71-72
...repository.ts	100	74.19	100	100	80,92,119,146-165
...repository.ts	100	87.5	100	100	33
presentation/http	100	87.5	100	100	
app.ts	100	80	100	100	64-68
...tp/controllers	98.78	75	100	100	
contexto.ts	83.33	50	100	100	8
...tion/http/dtos	100	83.33	100	100	
dtos.ts	100	83.33	100	100	18
...n/http/mappers	100	87.5	100	100	
...sos.mapper.ts	100	87.5	100	100	10
...tp/middlewares	93.54	88	95.45	94.31	
...middleware.ts	100	91.66	100	100	22
...middleware.ts	85	71.42	100	88.88	30-36
...middleware.ts	91.17	93.75	80	90.9	43-44,59
shared	100	88.88	100	100	
logger.ts	100	80	100	100	7
-----	-----	-----	-----	-----	-----

B.3 Seguridad contra la API desplegada

Generadas con docs/evidencias/generar-evidencias-api.sh. Los tokens aparecen recortados.

Evidencias de seguridad - http://3.151.57.252 - 2026-10-01T15:51:10Z

===== 1. Login correcto: access token JWT RS256 (15 min) + refresh token =====

```
$ POST /v1/auth/token (password)
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-RateLimit-Remaining-Minute: 29
X-RateLimit-Limit-Minute: 30
RateLimit-Remaining: 29
RateLimit-Limit: 30
RateLimit-Reset: 50
X-Request-Id: b95bf3bd-39ba-4b52-abbb-c0010eebdd38
Cache-Control: no-store
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: 7471633d4f205303380c894a1443e044
```

```
{
  "accessToken": "...eyJhbGciOiJS(ocortado)",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "refreshToken": "...rt_35c637bb4(ocortado)",
  "scope": "perfil:leer planes:leer planes:escribir aportes:escribir retiros:escribir movimientos:leer cuentas:leer"
}
```

Cabecera y claims del JWT (decodificados, sin la firma):

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "billetera-ahorro-auth"
}
{
  "roles": ["CLIENTE"],
  "scope": "perfil:leer planes:leer planes:escribir aportes:escribir retiros:escribir movimientos:leer cuentas:leer",
  "iat": 1790869871,
  "exp": 1790870771,
  "aud": "billetera-ahorro-api",
  "iss": "billetera-ahorro-auth",
  "sub": "6a1f0c2e-4b3d-4e5f-8a7b-1c2d3e4f5a60",
  "jti": "3d5810f8-8b4b-497e-a098-f265f9bc506a"
}
```

===== 2. Credenciales inválidas → 401 problem+json =====

```
$ POST /v1/auth/token (contraseña incorrecta)
sed: RE error: illegal byte sequence
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 27
X-RateLimit-Limit-Minute: 30
RateLimit-Remaining: 27
RateLimit-Limit: 30
RateLimit-Reset: 49
X-Request-Id: 4b423915-5ca3-4e32-ac0a-975f0c390715
```

===== 3. Sin token → 401 en el Gateway =====

```
$ GET /v1/planes-ahorro
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
WWW-Authenticate: Bearer
X-Request-Id: 2d6b8b8c-338a-4980-90eb-12c75d4be1bc
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Response-Latency: 1
Server: kong/3.9.3
X-Kong-Request-Id: 2b39713b5271c92d6160cd52722d2260
```

```
{
  "message": "Unauthorized"
}
```

===== 4. Token con firma falsificada → 401 =====

```
$ GET /v1/planes-ahorro (Authorization: Bearer <token alterado>)
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
WWW-Authenticate: Bearer error="invalid_token"
```

```

X-Request-Id: 3d6a9af1-2a15-426a-b4e0-9ba1c0b6f98b
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
    RateLimit-Remaining,RateLimit-Reset
X-Kong-Response-Latency: 1
Server: kong/3.9.3
X-Kong-Request-Id: 5780bd2983b90e4448ecd87fe077a104

{"message":"Invalid signature"}

===== 5. Scope insuficiente → 403 (OPERADOR intenta crear un plan) =====
$ POST /v1/planes-ahorro con token de OPERADOR
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 59999
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59999
RateLimit-Limit: 60000
RateLimit-Reset: 47
X-Request-Id: 9418a533-2690-4470-8c43-f9466a67b173
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
    RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: 4b1d452dca1e3d749651be77b406285e

{"type":"https://api.billetera-ahorro.example.com/problemas/permiso-insuficiente","title":"Permiso
insuficiente","status":403,"codigo":"PERMISO_INSUFICIENTE","detail":"Se requiere el scope planes:
escribir.","instance":"/v1/planes-ahorro"}

===== 6. Titularidad (ABAC): plan inexistente o ajeno → 404 =====
$ GET /v1/planes-ahorro/{id ajeno}
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 59998
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59998
RateLimit-Limit: 60000
RateLimit-Reset: 47
X-Request-Id: 85ae3bcc-98c4-435a-92bd-dd29230ff196
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
    RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: f92da314238a560568a2d8726e9f0652

{"type":"https://api.billetera-ahorro.example.com/problemas/no-encontrado","title":"Recurso no encontrado
","status":404,"codigo":"NO_ENCONTRADO","detail":"No existe el plan 61171500-26c3-40f7-9100-9
c5162bfae13.","instance":"/v1/planes-ahorro/61171500-26c3-40f7-9100-9c5162bfae13"}

===== 7. Validación del contrato → 400 con detalle por campo =====
$ POST /v1/planes-ahorro (cuerpo inválido)
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 59997
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59997
RateLimit-Limit: 60000
RateLimit-Reset: 46
X-Request-Id: 8b524861-dae9-437a-86ac-0d80a624d52f
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
    RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: 72aaa908a0ae613d82e2e61651f20867

{"type":"https://api.billetera-ahorro.example.com/problemas/validacion","title":"Datos inválidos","status
":400,"codigo":"VALIDACION","detail":"La petición tiene 6 error(es) de validación.","instance":"/v1/
planes-ahorro","errores":[{"campo":"nombre","mensaje":"Too small: expected string to have >=1
characters"}, {"campo":"icono","mensaje":"Invalid option: expected one of \"shield\"|\"flight_takeoff

```

```

\\|"directions_car"|"home"|"school"|"celebration"|"savings"|"favorite""},{ "campo":
montoMetaCentavos", "mensaje": "debe ser mayor o igual a 1"}, {"campo": "fechaObjetivo", "mensaje": "debe
ser una fecha YYYY-MM-DD"}, {"campo": "diaDebito", "mensaje": "Too big: expected number to be <=28"}, {"
campo": "cuentaDebitoId", "mensaje": "debe ser un UUID"}]]}

===== 8. Idempotency-Key: primera creación → 201 + Location + ETag =====
$ POST /v1/planes-ahorro (Idempotency-Key: 08b18214-5970-4b7a-acdb-c312fef68ac9)
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
X-RateLimit-Remaining-Minute: 59996
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59996
RateLimit-Limit: 60000
RateLimit-Reset: 46
X-Request-Id: fdf82812-b878-4a37-b24c-581a3daad719
Location: /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466
ETag: W/"v1"
Idempotent-Replayed: false
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: 37cefb2a4087700b9ca7813640c06969

{"id": "39548beb-8c34-413f-90e9-8936b9301466", "nombre": "Evidencia", "icono": "savings", "estado": "ACTIVO", "
moneda": "USD", "montoMetaCentavos": 120000, "fechaObjetivo": "2027-12-01", "cuotaMensualCentavos": 8315, "
plazoMeses": 14, "prorrogasMeses": 0, "diaDebito": 10, "cuentaDebito": {"id": "0c6f2a9e-1b3d-4e5f-8a7b-6
c5d4e3f2a1b", "tipo": "AHORROS", "alias": "pablo.ahorros", "moneda": "USD", "numeroEnmascarado
": "*****8901"}, "bloqueado": true, "bloqueadoDesde": "2026-10-01T15:51:14.783Z", "tasas": {"baseAnual
": 4.5, "bonoBloqueoAnual": 1.1, "totalAnual": 5.6, "efectivaAnual": 5.75}, "saldoCentavos": 0, "
saldoDisponibleCentavos": 0, "interesesDevengadosCentavos": 0, "progreso": 0, "proximoDebito": "2026-10-10", "
fechaInicio": "2026-10-01", "fechaFinEstimada": "2027-11-10", "creadoEn": "2026-10-01T15:51:14.783Z", "
actualizadoEn": "2026-10-01T15:51:14.783Z"}

===== 9. Idempotency-Key: reintento idéntico → misma respuesta, Idempotent-Replayed: true =====
$ POST /v1/planes-ahorro (misma clave y cuerpo)
HTTP/1.1 201 Created
location: /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466
Idempotent-Replayed: true

===== 10. Idempotency-Key reutilizada con otro cuerpo → 409 =====
$ POST /v1/planes-ahorro (misma clave, otro cuerpo)
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 59994
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59994
RateLimit-Limit: 60000
RateLimit-Reset: 45
X-Request-Id: cab85969-6f32-46ff-91ab-b2eb84de3983
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: 75131e553b4b6840a9c77c3bcc93db7e

{"type": "https://api.billetera-ahorro.example.com/problemas/idempotencia", "title": "Idempotency-Key
reutilizada", "status": 409, "codigo": "IDEMPOTENCIA_CONFLICTO", "detail": "La clave ya se usó con un cuerpo
distinto.", "instance": "/v1/planes-ahorro"}

===== 11. Patrón State: retiro en plan bloqueado → 409 RETIRO_NO_PERMITIDO =====
$ POST /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466/retiros
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 59993
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59993
RateLimit-Limit: 60000
RateLimit-Reset: 45

```



```

X-Request-Id: 267a095f-b05f-4173-98c9-30acafd3dac2
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: be047704f1b3553bad0b7d03865fa953

{"type":"https://api.billetera-ahorro.example.com/problemas/retiro-no-permitido","title":"Retiro no
permitido","status":409,"codigo":"RETIRO_NO_PERMITIDO","detail":"El plan tiene los fondos bloqueados.
Desbloquéelo antes de retirar.","instance":"/v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466/
retiros"}

===== 12. Concurrencia optimista: If-Match desactualizado → 412 =====
$ PATCH /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466 (If-Match: W/"v0")
HTTP/1.1 412 Precondition Failed
Content-Type: application/problem+json; charset=utf-8
X-RateLimit-Remaining-Minute: 59992
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59992
RateLimit-Limit: 60000
RateLimit-Reset: 45
X-Request-Id: 874815c1-7032-472b-a2cc-c39d5dcf96d7
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: 5a8b1c6ade6f84351e077a5da5a86117

{"type":"https://api.billetera-ahorro.example.com/problemas/version-desactualizada","title":"Versión
desactualizada","status":412,"codigo":"VERSION_DESACTUALIZADA","detail":"El plan cambió desde que se
leyó.","instance":"/v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466"}

===== 13. Aporte asíncrono → 202 + Location; luego EJECUTADO (DebitoSolicitado → Core → DebitoEjecutado)
=====
$ POST /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466/aportes
HTTP/1.1 202 Accepted
Content-Type: application/json; charset=utf-8
X-RateLimit-Remaining-Minute: 59991
X-RateLimit-Limit-Minute: 60000
RateLimit-Remaining: 59991
RateLimit-Limit: 60000
RateLimit-Reset: 44
X-Request-Id: 27255234-e8f5-4a14-9063-6eae50bd9d44
Location: /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466/aportes/2b2fd6c2-42c4-4f85-abe6-6
bc384e39ef5
Idempotent-Replayed: false
Server: kong/3.9.3
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location,ETag,Retry-After,X-Request-Id,Idempotent-Replayed,RateLimit-Limit,
RateLimit-Remaining,RateLimit-Reset
X-Kong-Request-Id: a88ec129b45838559af8975a2bc643e

{"id":"2b2fd6c2-42c4-4f85-abe6-6bc384e39ef5","planId":"39548beb-8c34-413f-90e9-8936b9301466","montoCentavos
":500,"moneda":"USD","cuentaOrigen":{"id":"0c6f2a9e-1b3d-4e5f-8a7b-6c5d4e3f2a1b","tipo":"AHORROS","
alias":"pablo.ahorros","moneda":"USD","numeroEnmascarado":"*****8901"},"estado":"PENDIENTE","
solicitadoEn":"2026-10-01T15:51:16.206Z","procesadoEn":null,"motivoFallo":null,"correlationId":"4
c45d2ec-50af-4538-ba30-d053c90c30eb"}

$ GET /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466/aportes/2b2fd6c2-42c4-4f85-abe6-6bc384e39ef5
(3 s después)
{"id":"2b2fd6c2-42c4-4f85-abe6-6bc384e39ef5","planId":"39548beb-8c34-413f-90e9-8936b9301466","montoCentavos
":500,"moneda":"USD","cuentaOrigen":{"id":"0c6f2a9e-1b3d-4e5f-8a7b-6c5d4e3f2a1b","tipo":"AHORROS","
alias":"pablo.ahorros","moneda":"USD","numeroEnmascarado":"*****8901"},"estado":"FALLIDO","
solicitadoEn":"2026-10-01T15:51:16.206Z","procesadoEn":"2026-10-01T15:51:16.887Z","motivoFallo":"
FONDOS_INSUFICIENTES","correlationId":"4c45d2ec-50af-4538-ba30-d053c90c30eb"}

===== 14. Limpieza: cancelación con devolución del saldo =====
$ POST /v1/planes-ahorro/39548beb-8c34-413f-90e9-8936b9301466/cancelacion
{'saldoCentavos': 0, 'interesesPerdidosCentavos': 0, 'montoDevueltoCentavos': 0, 'estadoDevolucion': '
EJECUTADO'} CANCELADO

```

```

===== 15. Refresh token con rotación: el usado deja de servir → 401 =====
$ POST /v1/auth/token (refresh_token, dos veces)
Primer uso:
HTTP 200
Reutilizado:
HTTP 401

===== 16. TLS: el Gateway negocia TLS 1.3 y rechaza TLS 1.2 =====
$ openssl s_client -connect 3.151.57.252:443
--- con -tls1_3:
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
--- con -tls1_2:
80A1E5F101000000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1 alert protocol version:ssl/record/
rec_layer_s3.c:918:SSL alert number 70
New, (NONE), Cipher is (NONE)
80A1E5F101000000:error:0A000197:SSL routines:SSL_shutdown:shutdown while in init:ssl/ssl_lib.c:2804:

===== 17. Endpoint público: caché del Gateway (X-Cache-Status) y rate limit (RateLimit-Remaining) =====
$ GET /v1/tarifas (dos veces)
HTTP/1.1 200 OK
RateLimit-Remaining: 599
Cache-Control: public, max-age=300
X-Cache-Status: Hit

HTTP/1.1 200 OK
RateLimit-Remaining: 598
Cache-Control: public, max-age=300
X-Cache-Status: Hit

===== 18. Rate limiting del login (30/min por IP) → 429 + Retry-After =====
$ 40 × POST /v1/auth/token con credenciales incorrectas
401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 401 429 429 429 429
429 429 429 429 429 429 429 429 429 429 429 429 429 429
HTTP/1.1 429 Too Many Requests
Retry-After: 25
RateLimit-Remaining: 0
RateLimit-Limit: 30
RateLimit-Reset: 25
"message":"API rate limit exceeded",

```

B.4 Despliegue en AWS

Recursos AWS del proyecto (etiqueta Proyecto=billetera-ahorro) - us-east-2 - 2026-10-01T15:50:03Z

```
== Instancia EC2
```

DescribeInstances	
AMI	ami-0fa99aa8f97f9e30b
Estado	running
IMDSv2	required
IP	3.151.57.252
Id	i-07b5f39c0f2cd5eec
Role	arn:aws:iam::<cuenta>:instance-profile/billetera-ahorro-ec2
Tipo	m7i-flex.large
Zona	us-east-2c

== Elastic IP

DescribeAddresses

IP	Instancia
3.151.57.252	i-07b5f39c0f2cd5eec

== Grupo de seguridad (reglas de entrada)

DescribeSecurityGroups			
Descripcion	Origen	Puerto	
HTTP	0.0.0.0/0	80	
HTTPS TLS 1.3	0.0.0.0/0	443	

== Secretos en SSM Parameter Store (solo nombres y tipo)

DescribeParameters	
Nombre	Tipo
/billetera-ahorro/JWT_PRIVATE_KEY	SecureString
/billetera-ahorro/JWT_PUBLIC_KEY	SecureString
/billetera-ahorro/MONGO_URI	SecureString
/billetera-ahorro/POSTGRES_PASSWORD	SecureString
/billetera-ahorro/RABBITMQ_PASSWORD	SecureString

== Imágenes en ECR (últimas 5)

DescribeImages			
MB	Subida	Tags	
68774224	2026-10-01T10:49:18.989000-05:00	manual-6	
1418	2026-10-01T10:49:18.595000-05:00	-	
68774224	2026-10-01T10:49:17.985000-05:00	-	
68774030	2026-10-01T10:47:24.671000-05:00	manual-5	
1418	2026-10-01T10:47:24.317000-05:00	-	

== Permisos del usuario del CI (billetera-ci)

GetUserPolicy	
Acciones	
Sid	
ecr:GetAuthorizationToken	LoginEcr
ecr:BatchCheckLayerAvailability, ecr:InitiateLayerUpload, ecr:UploadLayerPart, ecr:CompleteLayerUpload, ecr:PutImage, ecr:BatchGetImage, ecr:GetDownloadUrlForLayer	PublicarImagen
ec2:DescribeInstances, ssm:DescribeInstanceInformation	VerInstancias
ec2:StartInstances, ec2:StopInstances	EncenderApagarSoloBilletera
ssm:SendCommand	DesplegarSoloEnBilletera
ssm:SendCommand	DocumentoShell
ssm:GetCommandInvocation, ssm:ListCommandInvocations	ResultadoDespliegue

== docker compose ps

SERVICE	IMAGE	PORTS	STATUS
api	<cuenta>.dkr.ecr.us-east-2.amazonaws.com/billetera-ahorro-backend:manual-6		Up 39 seconds

```

auth          3000-3001/tcp, 4000/tcp
               <cuanta>.dkr.ecr.us-east-2.amazonaws.com/billetera-ahorro-backend:manual-6   Up 39 seconds
batch         3000-3001/tcp, 4000/tcp
               <cuanta>.dkr.ecr.us-east-2.amazonaws.com/billetera-ahorro-backend:manual-6   Up 38 seconds
core-adapter  3000-3001/tcp, 4000/tcp
               <cuanta>.dkr.ecr.us-east-2.amazonaws.com/billetera-ahorro-backend:manual-6   Up 39 seconds
core-simulado 3000-3001/tcp, 4000/tcp
               <cuanta>.dkr.ecr.us-east-2.amazonaws.com/billetera-ahorro-backend:manual-6   Up 40 seconds
kong          kong:3.9
               seconds (healthy) 8001/tcp, 8444/tcp, 0.0.0.0:80->8000/tcp, [::]:80->8000/tcp, 0.0.0.0:443->8443/tcp
               , [::]:443->8443/tcp
postgres      postgres:16-alpine
               minutes (healthy) 5432/tcp
               Up 3
rabbitmq      rabbitmq:4-management-alpine
               minutes (healthy) 4369/tcp, 5671-5672/tcp, 15671-15672/tcp, 15691-15692/tcp, 25672/tcp
               Up 3

== docker stats (reposo)
NAME          CPU %      MEM USAGE / LIMIT
billetera-batch-1    0.13%     29.84MiB / 7.594GiB
billetera-kong-1     11.29%    167.9MiB / 7.594GiB
billetera-auth-1     0.00%     36.02MiB / 7.594GiB
billetera-core-adapter-1 0.03%     25.86MiB / 7.594GiB
billetera-api-1      0.39%     39.67MiB / 7.594GiB
billetera-core-simulado-1 0.14%     48.01MiB / 7.594GiB
billetera-postgres-1 0.18%     51.45MiB / 7.594GiB
billetera-rabbitmq-1 0.18%     112.9MiB / 7.594GiB

== Sistema
Linux 7.0.0-1013-aws x86_64
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1

```

B.5 Pipeline de CI/CD

Ejecución del pipeline Backend CI/CD - run 36812583029 (commit 547fcb3)
<https://github.com/pajmunoz/Trabajo-Final-ArquitecturaApis/actions/runs/36812583029>

```

JOB Pruebas y contrato: success (2026-10-01T03:53:39Z → 2026-10-01T03:54:12Z)
  success Set up job
  success Initialize containers
  success Run actions/checkout@v4
  success Run actions/setup-node@v4
  success Run npm ci
  success Tipos (TypeScript)
  success Contrato OpenAPI válido
  success Pruebas unitarias e integración con cobertura (umbral 80 %)
  success Resumen de cobertura
  success Run actions/upload-artifact@v4
  success Post Run actions/setup-node@v4
  success Post Run actions/checkout@v4
  success Stop containers
  success Complete job
JOB Configuración: success (2026-10-01T03:53:39Z → 2026-10-01T03:53:42Z)
  success Set up job
  success Run if [ -n "$CLAVE" ]; then echo "aws=true" >> "$GITHUB_OUTPUT"; else
  success Complete job
JOB Imagen en ECR: success (2026-10-01T03:54:14Z → 2026-10-01T03:54:56Z)
  success Set up job
  success Run actions/checkout@v4
  success Run aws-actions/configure-aws-credentials@v4
  success Run aws-actions/amazon-ecr-login@v2
  success Run echo "imagen=<cuanta>.dkr.ecr.us-east-2.amazonaws.com/billetera-ahorro-backend:${GITHUB_SHA:~7}" >> "$GITHUB_OUTPUT"
  success Run docker/setup-buildx-action@v3
  success Run docker/build-push-action@v6
  success Post Run docker/build-push-action@v6
  success Post Run docker/setup-buildx-action@v3
  success Post Run aws-actions/amazon-ecr-login@v2

```

```
success Post Run aws-actions/configure-aws-credentials@v4
success Post Run actions/checkout@v4
success Complete job
JOB Despliegue en EC2: success (2026-10-01T03:54:59Z → 2026-10-01T03:55:05Z)
success Set up job
success Run aws-actions/configure-aws-credentials@v4
success Ubicar la instancia
skipped Encender la instancia (solo si se pidió)
success Instancia apagada
skipped Desplegar con SSM Run Command
skipped Prueba de humo pública
success Post Run aws-actions/configure-aws-credentials@v4
success Complete job
```

APÉNDICE C

Distribución del trabajo

Pendiente

Completar con los integrantes y las responsabilidades de cada uno. La rúbrica valora una división equitativa y que todos puedan defender cualquier parte del proyecto.

Tabla C.1: *Distribución del trabajo por integrante*

Integrante	Responsabilidades principales